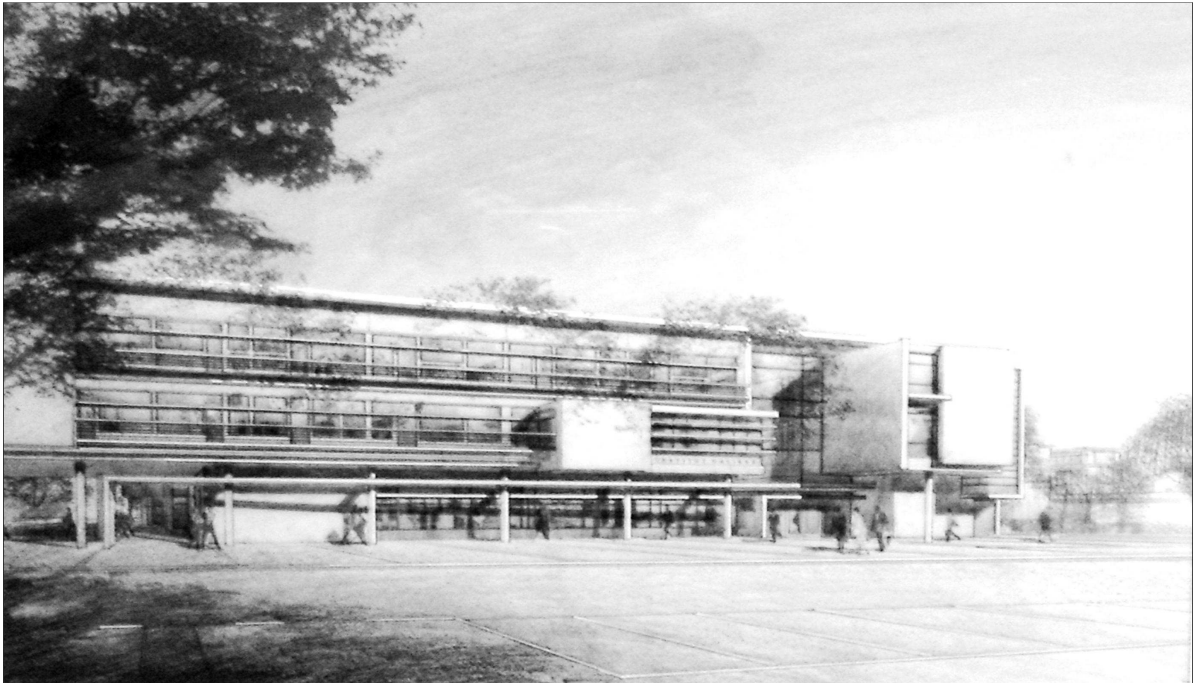




ASPECTS MATERIELS & LOGICIELS DU MICROCONTRÔLEUR PIC 16F877

RESUME DU COURS PIC

THIEBAUT Bernard

Microprocesseurs/Microcontrôleurs

Ce n'est pas compliqué : **il y en a partout !!!**

Quelques exemples (liste non exhaustive) :

- l'électronique embarquée :

en automobile (ABS, injection électronique, tableaux de bord, régulateur de vitesse, systèmes anti-vol, GPS...), dans les terminaux portables de paiement, la petite robotique ...

- la domotique, *(maison intelligente...)*

- les téléphones portables

et autres applications portables type MP3...

- les appareils photos numériques,

- l'électroménager :

téléviseurs, télécommandes diverses, machines à laver, fours à micro ondes, cafetières...

- les jouets :

véhicules télécommandés, système légo Mindstorms...

- les applications spécifiques et/ou délocalisées :

distributeurs divers...

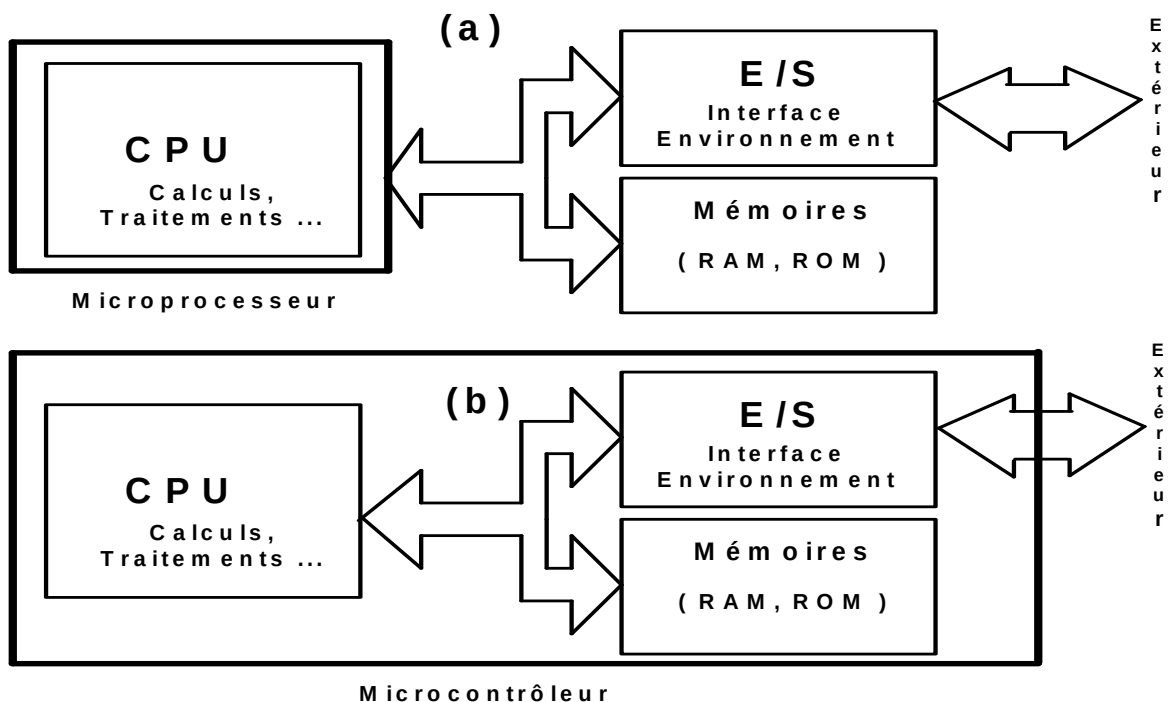
- les petits automatismes industriels,

- la sécurité, la signalisation et la protection,

- et bien d'autres applications ...

Microprocesseurs/Microcontrôleurs (suite)

Principales différences



CPU : Central Process Unit :
là où sont faits les divers traitements
C'est le « cœur » des micro/processeurs/contrôleurs

Figure I.1. (page 6)

Quelques exemples d'E/S

- **E/S parallèles** $\Rightarrow$ capteurs ou d'actionneurs logiques,
- **timers** $\Rightarrow$ gestion du temps ou compteurs d'événements extérieurs,
- **CAN ou CNA** $\Rightarrow$ capteurs ou actionneurs analogiques,
- **E/S série synchrones ou asynchrones** $\Rightarrow$ communications,
- **autres modules** $\Rightarrow$ suivant les utilisations et applications...

Objectifs principaux de ces cours /TP

Comprendre pourquoi et comment marche un microcontrôleur.

Savoir l'utiliser pour le mettre en œuvre dans une application.

Principales difficultés

Une **porte logique** :

⇒ documentation sur une page

Un **circuit numérique de base** (comparateur, compteur...)

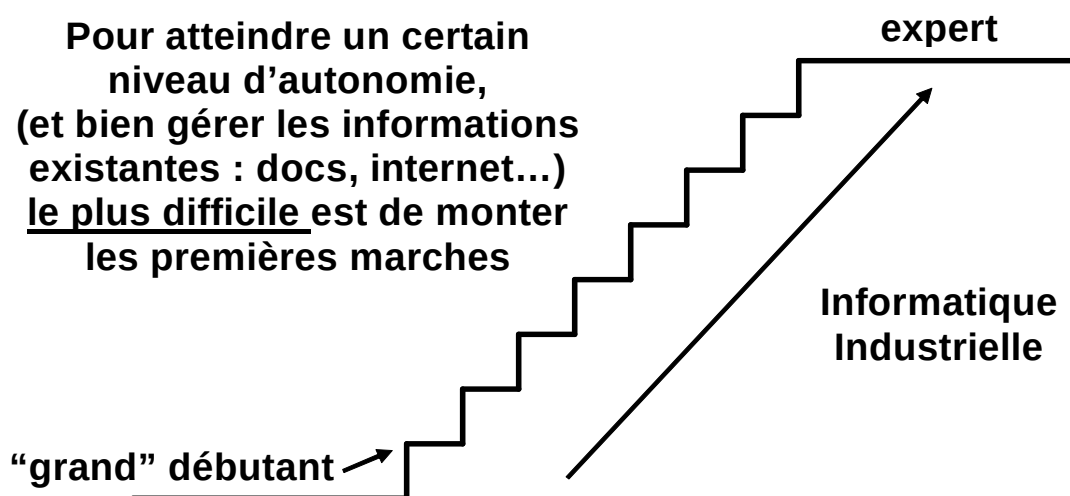
⇒ documentation sur quelques pages

Un **microcontrôleur** : inflation de la doc !!!

⇒ documentation sur plusieurs centaines de pages !!!

*le plus souvent en **anglais** et surtout ...*

pas forcément faite pour des débutants !!!



**Cinq grandes parties dans le polycopié
de cours complet (462 pages)
(seules les parties ① et ② seront résumées ici)**

① Chapitre I

Généralités et notions de base.

Grands principes de fonctionnement.

② Chapitres II, III et IV

Spécificités du microcontrôleur PIC 16F877A utilisé en TP.
(architecture interne, fonctionnement et jeu d'instructions).

③ Chapitres V et VI

Exercices corrigés.

Exemples de programmes de base en assembleur et en C.

④ Chapitre VII

Méthodologie de travail en TP.

⑤ Chapitre VIII

Ressources du **PIC** comme interface avec son environnement
(cf. le document **Résumé du cours pendant TP PIC.pdf**)

**Mais, si vous comprenez bien toutes les notions présentées ici,
vous n'aurez aucun mal à les transposer
sur tous les PIC commercialisés par MICROCHIP.
Mais aussi, plus généralement,
sur tous les types de micro/processeurs/contrôleurs.**

Résumé du cours PIC : chapitre I

1. Bases logique **combinatoire** et différentes **portes logiques**.

Entrées		INVERSION		ET (AND)	OU Inclusif (IOR)	OU eXclusif (XOR)
A	B	$\bar{A}$	$\bar{B}$	$A \cdot B$	$A + B$	$A \oplus B$
0	0	1	1	0	0	0
0	1	1	0	0	1	1
1	0	0	1	0	1	1
1	1	0	0	1	1	0

2. Bases logique **séquentielle** :

possibilité de mémorisation d'un **bit** (0 ou 1) dans une **bascule**.

3. Les **registres** : il y en a partout !!!

Regroupement de $n = 2^k$ bascules souvent représenté par :

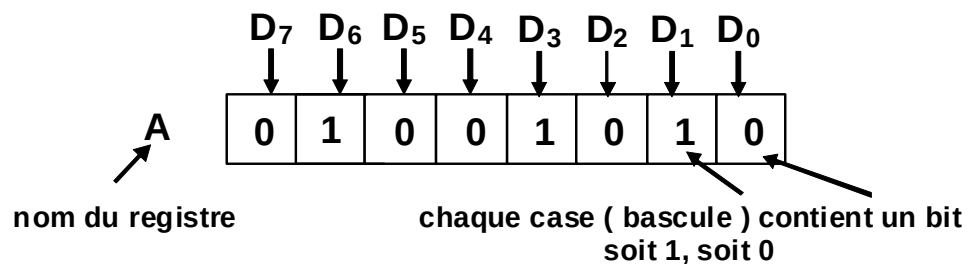


Schéma standard d'un registre pour $n = 8$

Figure I.2. (page 9)

Taille (valeur de n)	Nom	Name
4	Quartet	Nibble
8	Octet	Byte
16	Mot	Word
32	Double mot	Double
...		

Résumé du cours PIC : chapitre I

La suite des **n** bits **D_{n-1} D_{n-2} ... D₁ D₀** (chacun valant soit **0** soit **1**) représente le **codage binaire** (en **base 2**) de l'information contenue dans le registre.

Sa valeur en **décimal** (base 10) s'exprime par :

$$D_{n-1}.2^{n-1} + D_{n-2}.2^{n-2} + ... + D_2.2^2 + D_1.2^1 + D_0.2^0$$

D₀ est le bit de poids **faible** (Less Significant Bit ou **LSB**)

D_{n-1} est le bit de poids **fort** (Most Significant Bit ou **MSB**)

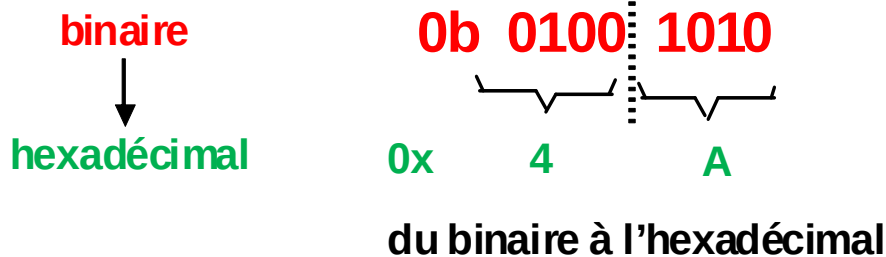
En pratique, le codage **hexadécimal** (en **base 16**) est plus utilisé.

Codage décimal	Codage binaire	Codage hexadécimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Figure I.3. (page11)

Correspondance **binaire** ↔ **hexadécimal**

Exemple pour un octet



Exemple pour un mot

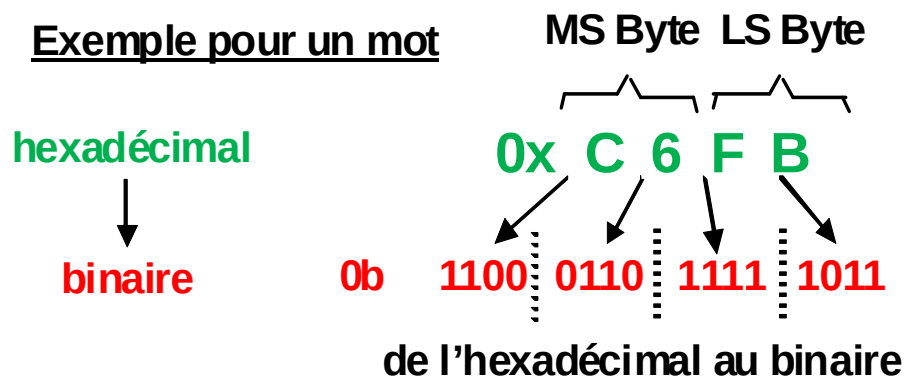


Figure I.4. (page 12)

**Règle de passage à connaître parfaitement :
très utile pour les masques,
le repérage des bits dans un registre,
la mise en œuvre et l'utilisation des registres,
la lecture de la documentation...**

Résumé du cours PIC : chapitre I

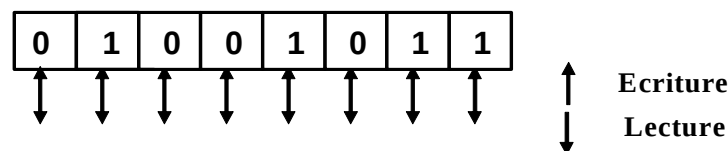
Principales propriétés d'un registre

Accès en **lecture** (cycle de lecture ou **Read Cycle**) :
pour **accéder** au **contenu** du registre.

Accès en **écriture** (cycle d'écriture ou **Write Cycle**) :
pour **modifier** le **contenu** du registre.

Deux modes d'accès à un registre

① le mode dit **parallèle**



Accès en **parallèle** : tous les bits du registre
sont lus ou modifiés en même temps

Figure I.5. (page14)

② le mode dit **série**

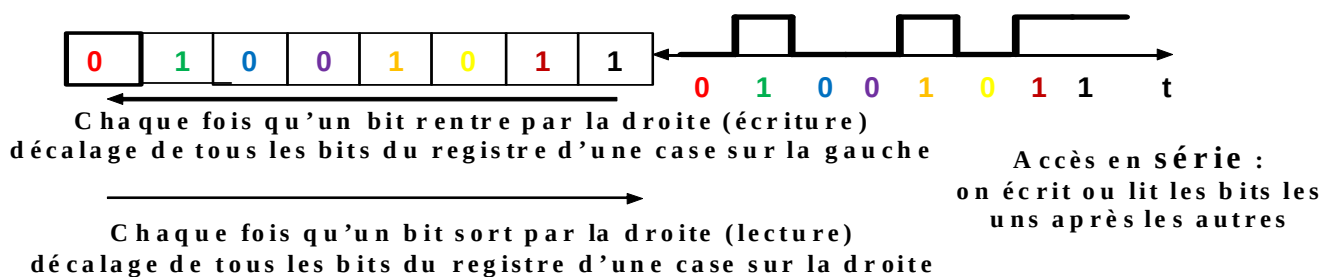


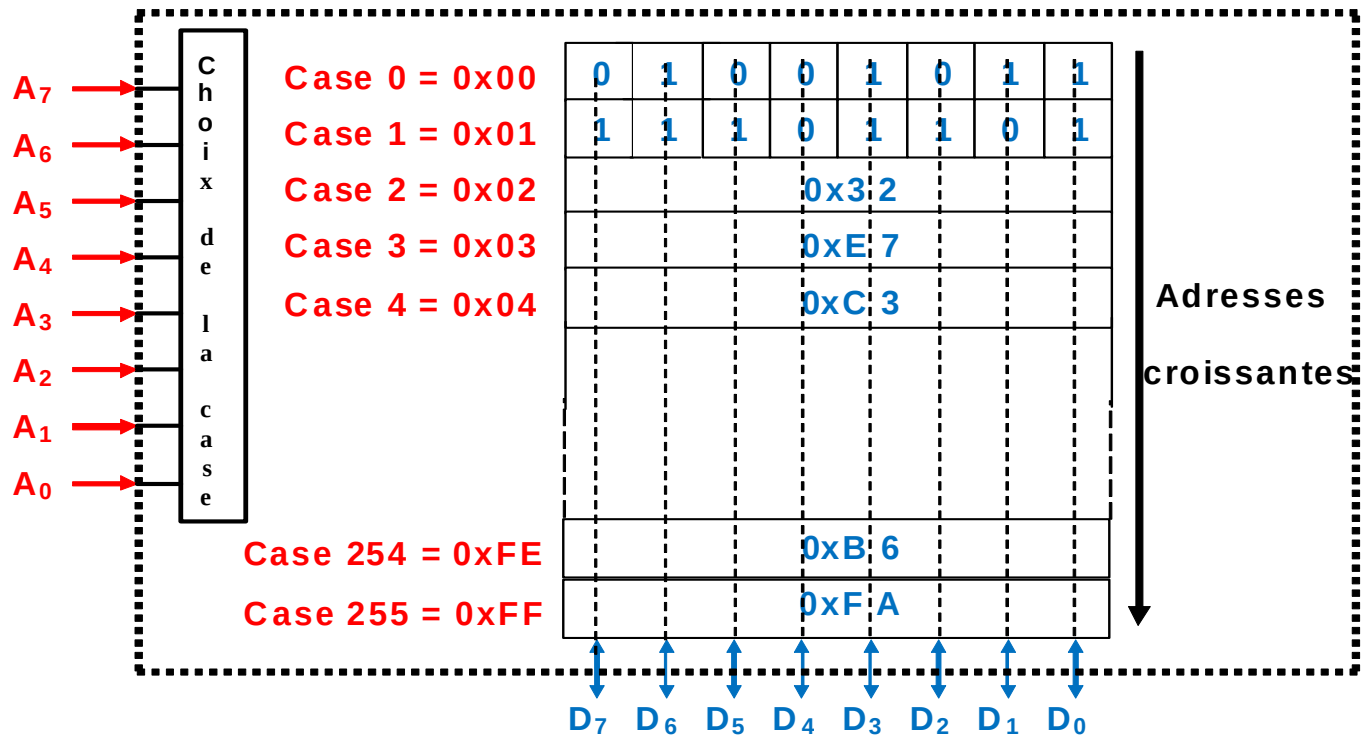
Figure I.6. (page 14)

	Avantages	Inconvénients
Mode parallèle	Plus rapide (1 seul coup d'horloge)	Connectique importante (n+1) fils
Mode série	Connectique simple 2 (ou 3 fils)	Plus lent (n coups d'horloge)

4. Les mémoires vives : ensemble de registres

pour stocker les programmes et les données.

Représentation « classique » d'une mémoire



Représentation "classique" d'une zone de mémoire vive.
Ici d'une taille (très modeste) de 256 octets.

Figure I.7. (page 17)

💣 💣 💣 **Attention !!!**

Ne pas confondre (source d'erreurs fréquentes):

le nombre « inscrit à côté de la case » :
son numéro ou plutôt son adresse, l'adresse de la donnée,
avec

le nombre « à l'intérieur de la case » :
son contenu, la valeur de la donnée.

Résumé du cours PIC : chapitre I

Les échanges avec la CPU

La combinaison binaire présente sur les lignes **d'entrée A_i** reflète le numéro (l'adresse) de la case mémoire concernée par l'échange (en lecture ou en écriture) entre la CPU et la mémoire. Par exemple si on a :

	A7	A6	A5	A4	A3	A2	A1	A0
binaire	0	1	0	1	0	1	1	1
hexadécimal	5				7			

la case concernée par l'échange est celle d'adresse **$0x57 = (87)_{10}$**

Le **contenu** de la case mémoire adressée (et elle seule !!!) sera ensuite échangé (en lecture ou en écriture) sur les lignes **D_i**

Remarque

Si on a **$r+1$** lignes d'adresses (donc de **A_0** à **A_r**), cela correspond à une **capacité totale** mémoire de **2^{r+1}** cases différentes.

Tailles mémoire et unités de mesure fréquentes

Puissance de 2 : capacité exacte	$\approx$ Puissance de 10	Arrondi à
$2^{10} = 1\,024$	$\approx$ 10^3	1 K (ilo)
$2^{20} = 1\,048\,576$	$\approx$ 10^6	1 M (éga)
$2^{30} = 1\,073\,741\,824$	$\approx$ 10^9	1 G (iga)
$2^{40} \approx 1,099511627 \cdot 10^{12}$	$\approx$ 10^{12}	1 T (éra)

Résumé du cours PIC : chapitre I

Les différents types de mémoires vives

Propriétés	RAM	ROM
Mode d'accès	Aléatoire	Aléatoire
Type d'accès	Lecture <u>et</u> écriture	Lecture <u>seulement</u>
Coupure alimentation	Volatile	Non volatile
Surtout pour :	Les données	Le programme (instructions)
Vitesse (cf. temps d'accès)	Rapides (surtout RAM statiques)	Plus lentes que les RAM
Capacité pour : $A_r, A_{r-1} \dots A_1 A_0$	2^{r+1} cases mémoire	2^{r+1} cases mémoire

Les différents types de ROM

Type de ROM	Principales Propriétés
ROM masquées	code dans masques fabrication en usine (grandes quantités, non modifiables)
ROM OTP	une seule fois programmable (technique des fusibles)
EPROM	effaçable aux UV re-programmable sur un programmeur
FLASH EEPROM	effaçable et re-programmable électriquement (sur la carte applicative)

En pratique

Ce sont les **RAM** et les **FLASH EEPROM**
qui sont les plus utilisées.

Attention :
**ROM et RAM ne contiennent pas
le même type d'information !!!**

La **ROM** contient le programme, c'est-à-dire la liste des **instructions élémentaires** que doit exécuter la CPU.

Mais qu'est-ce qu'une **instruction élémentaire** ?

**Une instruction élémentaire est la plus petite tâche
que peut effectuer une CPU :**

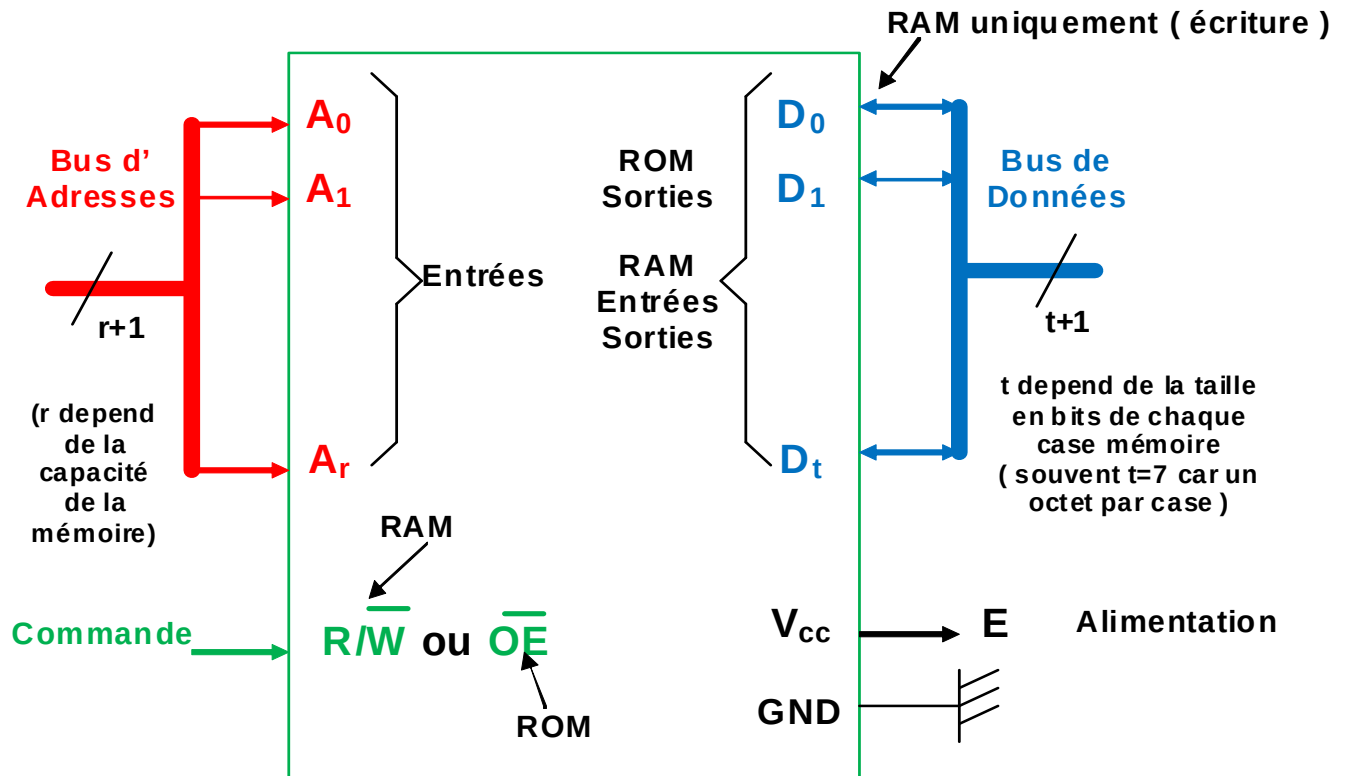
**comme un simple transfert
comme une simple addition...**

**Le jeu d'instructions est la liste exhaustive de toutes
les instructions élémentaires que sait effectuer cette CPU.**

La **RAM** contient les **données**, c'est-à-dire les **variables qui
sont concernées par chacune de ces instructions.**
Ce sont les **opérandes** des instructions.

**comme les cases mémoires qui sont concernées par un
transfert (source et destination)
comme les cases mémoires qui sont concernées par une
addition...**

Conclusion : une mémoire « vue de l'extérieur »



Brochage "classique"
d'une mémoire vive
(R.A.M. ou R.O.M.)

Figure I.8. (page 22)

Pour une RAM

Signal de lecture/écriture noté souvent : $\overline{R/W}$ avec :

- $\overline{R/W} = 1$: lecture (READ) d'une case RAM (sélectionnée par les A_i),
- $\overline{R/W} = 0$: écriture (WRITE) dans une case RAM (sélectionnée par les A_i).

Pour une ROM

Signal de validation de la sortie des données (lecture) : $\overline{OE}$ avec :

- $\overline{OE} = 1$: contenu case ROM (sélectionnée par les A_i) non présent sur les D_i ,
- $\overline{OE} = 0$: contenu case ROM (sélectionnée par les A_i) présent sur les D_i .

Résumé du cours PIC : chapitre I

Exemples d'écrans de visualisation du contenu de la mémoire (cf. TP)

Contenu « brut » en hexadécimal/ASCII
(sans référence symbolique)

avec affichage du contenu de
16 cases mémoire consécutives par ligne

Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
000	--	FF	18	1C	2A	10	01	98	00	00	00	00	00	00	C5	40	-...*...
010	00	00	00	FF	00	01	40	00	00	00	00	28	20	00	00	00	@. ... (...
020	00	00	A0	00	2B	00	00	00	08	28	12	0C	02	20	02	20	+... .(.... .
030	00	01	00	05	00	00	02	00	80	04	20	10	20	01	00	00	
040	10	00	00	40	20	00	18	06	00	00	00	0A	00	04	00	08	...@
050	2B	00	08	00	20	08	00	08	00	08	80	11	0E	20	08	00	+...
060	02	10	38	81	09	12	00	10	10	20	00	00	42	00	01	81	..8..... . ..B...
070	RR	E0	00	00	08	80	10	34	12	41	00	44	00	08	40	40	R.....4 .A.D..@@
080	--	FF	7F	1C	2A	3F	BF	FF	07	00	00	00	00	00	--	--	-...*?...
090	--	00	FF	00	00	--	--	--	02	00	--	--	07	00	00	00	-...--- ..--....
0A0	00	80	04	00	11	08	10	00	C0	58	80	00	20	02	00	00	X.. ...
0B0	00	01	10	00	00	00	4A	40	40	08	C0	60	08	22	41	C2	J@ @..`"A.
0C0	40	80	64	20	06	02	00	42	00	00	01	00	00	40	41	20	@.d ...B@.
0D0	00	80	15	00	10	C0	20	00	04	18	00	01	08	14	00	00	
0E0	00	00	00	06	00	04	04	30	10	80	00	40	00	20	00	00	0 ...@. ...
0F0	RR	E0	00	00	08	80	10	34	12	41	00	44	00	08	40	40	R.....4 .A.D..@@
100	--	FF	7F	1C	2A	--	01	--	--	--	00	00	E3	B8	3D	1C	-...*-,- --...=.
110	00	10	80	00	10	80	00	08	00	00	25	80	00	00	00	00	&.....

Exemple : contenu de la case **0x0BE** = **0x41** = code ASCII de A

Résumé du cours PIC : chapitre I

Contenu sous différents formats avec référence symbolique

avec affichage d'une case mémoire par ligne : ①

(contenu en hexa, en décimal, en binaire et en ASCII : ②)

et

nom du registre (documentation PIC : ③)

ou

de la variable (déclarée dans votre programme source)

① ← ② → ③ →

Address	Hex	Decimal	Binary	Char	Symbol Name
000	--	-	-----	-	INDF
001	0xFF	255	11111111	'.'	TMR0
002	0x18	24	00011000	'.'	PCL
003	0x1C	28	00011100	'.'	STATUS
004	0x2A	42	00101010	'*'	FSR
005	0x10	16	00010000	'.'	PORTA
006	0x01	1	00000001	'.'	PORTB
007	0x98	152	10011000	'.'	PORTC
008	0x00	0	00000000	'.'	PORTD
009	0x00	0	00000000	'.'	PORTE
00A	0x00	0	00000000	'.'	PCLATH
00B	0x00	0	00000000	'.'	INTCON
00C	0x00	0	00000000	'.'	PIR1
00D	0x00	0	00000000	'.'	PIR2
00E	0xC5	197	11000101	'.'	TMR1
00F	0x40	64	01000000	'@'	TMR1H
010	0x00	0	00000000	'.'	T1CON
011	0x00	0	00000000	'.'	TMR2

Hex Symbolic

Résumé du cours PIC : chapitre I

Quelques rappels sur le codage ASCII

Pour **coder** les caractères **imprimables** (ou affichables) :
les lettres, les chiffres, les signes de ponctuation...

TABLE DES CODES ASCII POUR LES 128 PREMIERS CARACTERES (MSB = 0)

Colonnes 1 et 2		Colonnes 3 à 8					
00 NUL	10 DLE	20	30	40	50	60	70
^@ 0	^P 16		0	@	P	`	p
		32	48	64	80	96	112
01 SOH	11 DC1	21	31	41	51	61	71
^A 1	^Q 17	!	1	A	Q	a	q
		33	49	65	81	97	113
02 STX	12 DC2	22	32	42	52	62	72
^B 2	^R 18	"	2	B	R	b	r
		34	50	66	82	98	114
03 ETX	13 DC3	23	33	43	53	63	73
^C 3	^S 19	#	3	C	S	c	s
		35	51	67	83	99	115
04 EOT	14 DC4	24	34	44	54	64	74
^D 4	^T 20	\$	4	D	T	d	t
		36	52	68	84	100	116
05 ENQ	15 NAK	25	35	45	55	65	75
^E 5	^U 21	%	5	E	U	e	u
		37	53	69	85	101	117
06 ACK	16 SYN	26	36	46	56	66	76
^F 6	^V 22	&	6	F	V	f	v
		38	54	70	86	102	118
07 BEL	17 ETB	27	37	47	57	67	77
^G 7	^W 23	'	7	G	W	g	w
		39	55	71	87	103	119
08 BS	18 CAN	28	38	48	58	68	78
^H 8	^X 24	(	8	H	X	h	x
		40	56	72	88	104	120
09 HT	19 EM	29	39	49	59	69	79
^I 9	^Y 25	)	9	I	Y	i	y
		41	57	73	89	105	121
0A LF	1A SUB	2A	3A	4A	5A	6A	7A
^J 10	^Z 26	*	:	J	Z	j	z
		42	58	74	90	106	122
0B VT	1B ESC	2B	3B	4B	5B	6B	7B
^K 11	^[27	+	;	K	[	k	{
		43	59	75	91	107	123
0C FF	1C FS	2C	3C	4C	5C	6C	7C
^L 12	^\ 28	,	<	L	\	l	!
		44	60	76	92	108	124
0D CR	1D GS	2D	3D	4D	5D	6D	7D
^M 13	^] 29	-	=	M	]	m	}
		45	61	77	93	109	125
0E SO	1E RS	2E	3E	4E	5E	6E	7E
^N 14	^^ 30	.	>	N	^	n	~
		46	62	78	94	110	126
0F SI	1F US	2F	3F	4F	5F	6F	7F
^O 15	^_ 31	/	?	O	_	o	
		47	63	79	95	111	127

Figure I.27. (page 55)

Résumé du cours PIC : chapitre I

Interprétation du contenu de chaque case :

C'est facile : exemple pour le code ASCII du 'A'

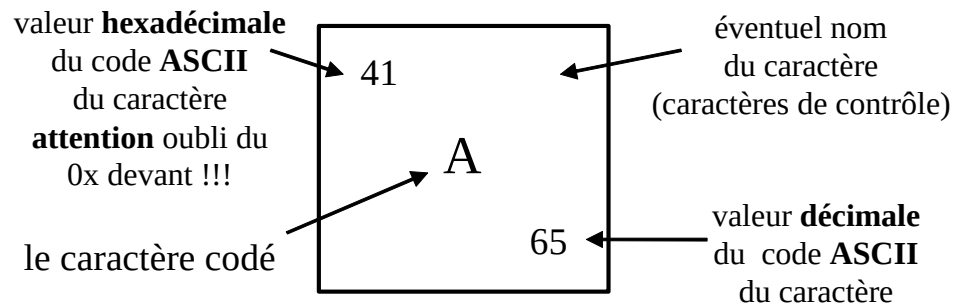


Figure I.26. (page 54)

Exemple de déclaration d'un caractère en langage C :

`char car = 'A' ;` // idem `char car = 0x41` (ou `65`) mais plus lisible

💣* **Mais attention pour le codage des chiffres !!!**

Ne pas confondre :

0x32 = '2' : code ASCII du 2 en hexadécimal

avec

0x02 = 2 : la valeur 2.

Pour afficher le chiffre 2 sur un terminal en mode texte, il faut lui envoyer 0x32 et non 0x02 !!!

Par contre, pour faire une addition avec le chiffre 2, il faut utiliser 0x02 et non 0x32 !!!

Tout dépend donc du contexte d'utilisation !!!

5. Les différents **bus**

pour **relier** les différents composants (CPU, mémoires, E/S...)

① Bus d'adresses (ADDRESS BUS)

chemin physique, constitué des lignes **A_i**,
sur lesquelles les adresses
(des différentes cases mémoire) seront envoyées
par la CPU vers les mémoires.
BUS MONODIRECTIONNEL : SENS CPU VERS MEMOIRE

② Bus de données (DATA BUS)

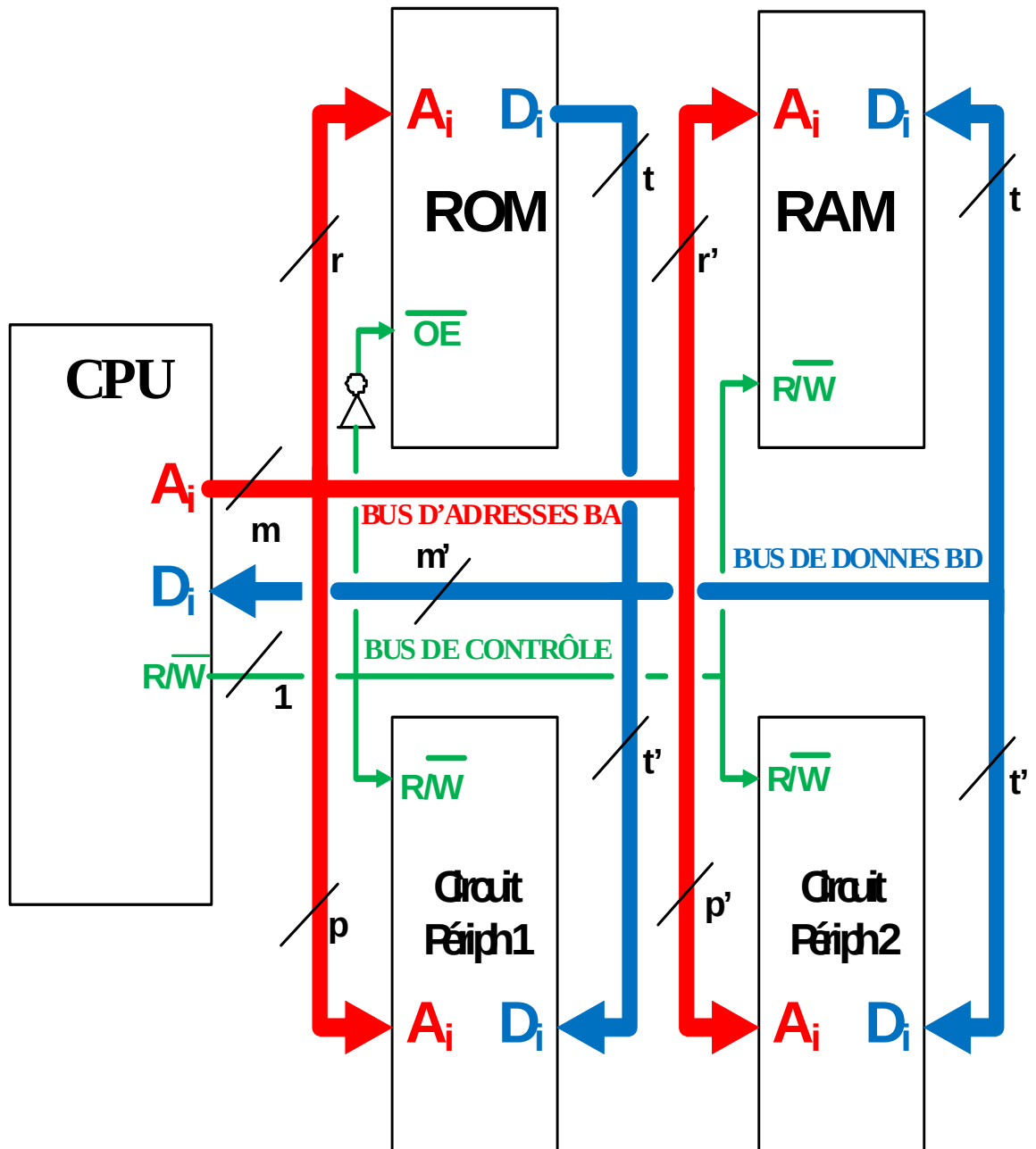
chemin physique, constitué ici des lignes **D_i**,
sur lesquelles les données
(contenu des cases mémoire) seront échangées
entre les mémoires et la CPU.
BUS BIRECTIONNEL

③ Bus de contrôle (CONTROL BUS)

chemin physique regroupant les différents signaux nécessaires
au fonctionnement du système complet
(synchronisations, horloges, lecture/écriture, reset...)

Synthèse

Exemple de synoptique simplifié d'une architecture



**Architecture « type » dite de Von Neumann
(on en verra les limites !!!)**

Figure I.11. (page 29)

Limitation de l'architecture de type Von Neumann

Dans une architecture Von Neumann, **ROM** et **RAM** partagent :

- un seul bus d'adresses **BA**,
- un seul bus de données **BD**.

D'où l'existence de « goulots d'étranglement » (cf aiguillages)

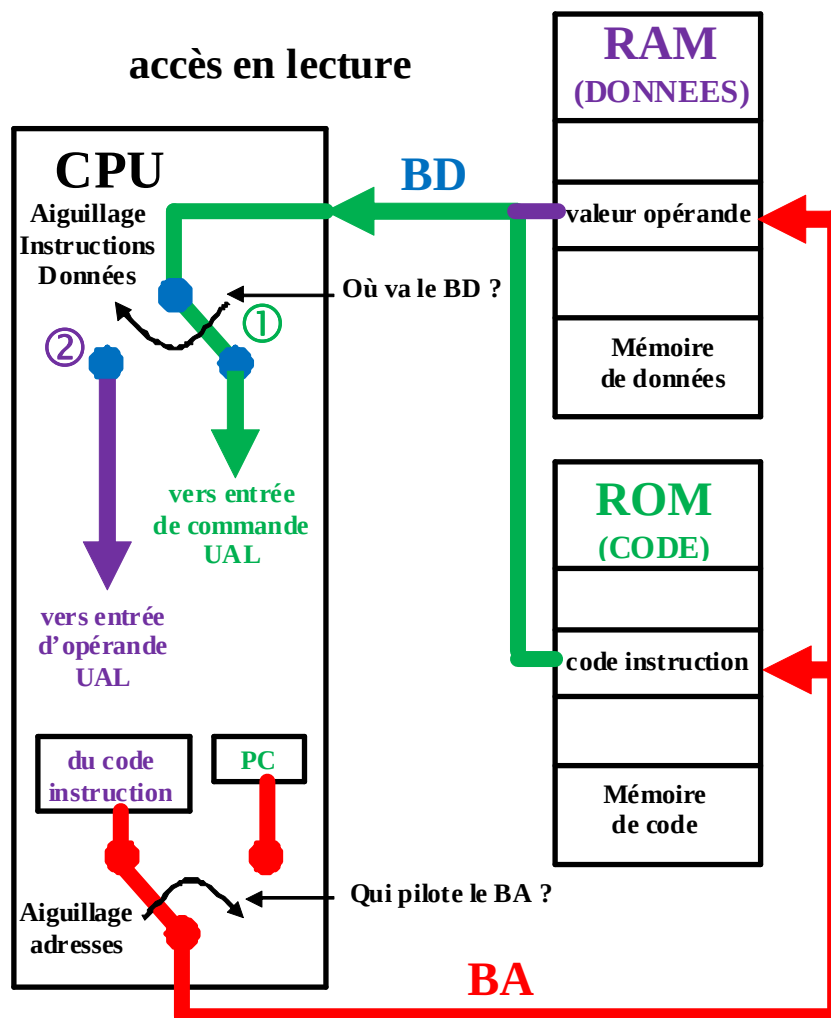


Figure I.19. (page 45)

Pourquoi ces aiguillages sur le **BD** et sur le **BA** ?

Pour apporter des éléments de réponse, il faut rentrer à l'intérieur de la CPU :

En effet, au cœur de toute **CPU** (microcontrôleur/processeur), il existe une **UAL**, qui effectue tous les **traitements** sur les **données**.

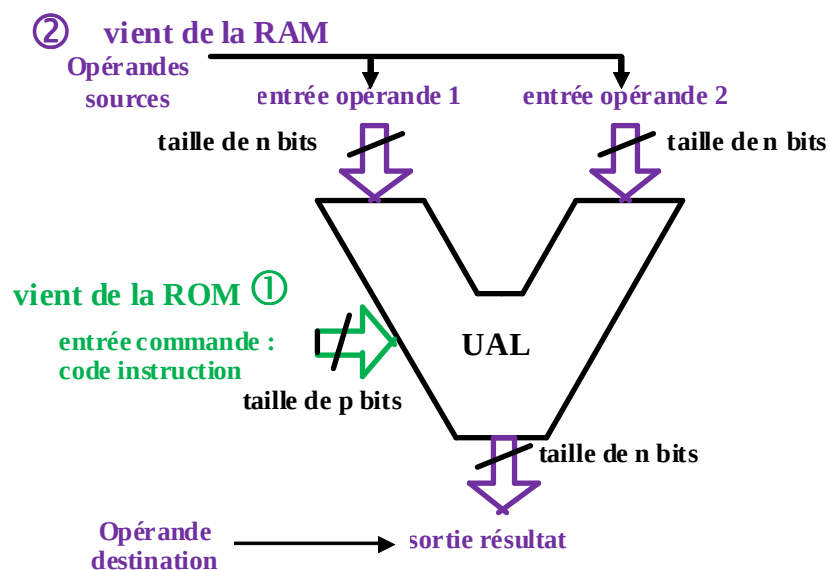


Figure I.20. (page 46)

Comme, les informations dans la **ROM (instructions)** et celles dans la **RAM (données)** ne sont **pas de même nature**, elles seront donc gérées en interne **différemment**.

D'où l'existence d'un **premier aiguillage** sur le **BD**.
(par **mutiplexage temporel**) et donc une **première perte de temps**.

Idem pour le bus d'adresse **BA** qui véhicule :

- soit **l'adresse de l'instruction** (dans **ROM**)
- soit **l'adresse de l'opérande** traitée par l'instruction (dans **RAM**)

D'où l'existence d'un **deuxième aiguillage** sur le **BA**
(par **mutiplexage temporel**) et donc une **deuxième perte de temps**.

Vers une architecture de type Harvard

Mémoire de données et de code complètement découplées

Et donc, au total, **4 bus indépendants** :

- un bus **BAC** : **B**us d'**A**dresses pour la mémoire de **C**ode (**ROM**)
- un bus **BAD** : **B**us d'**A**dresses pour la mémoire de **D**onnées (**RAM**)
- un bus **BDC** : **B**us de **D**onnées pour la mémoire de **C**ode (**ROM**)
- un bus **BDD** : **B**us de **D**onnées pour la mémoire de **D**onnées (**RAM**)

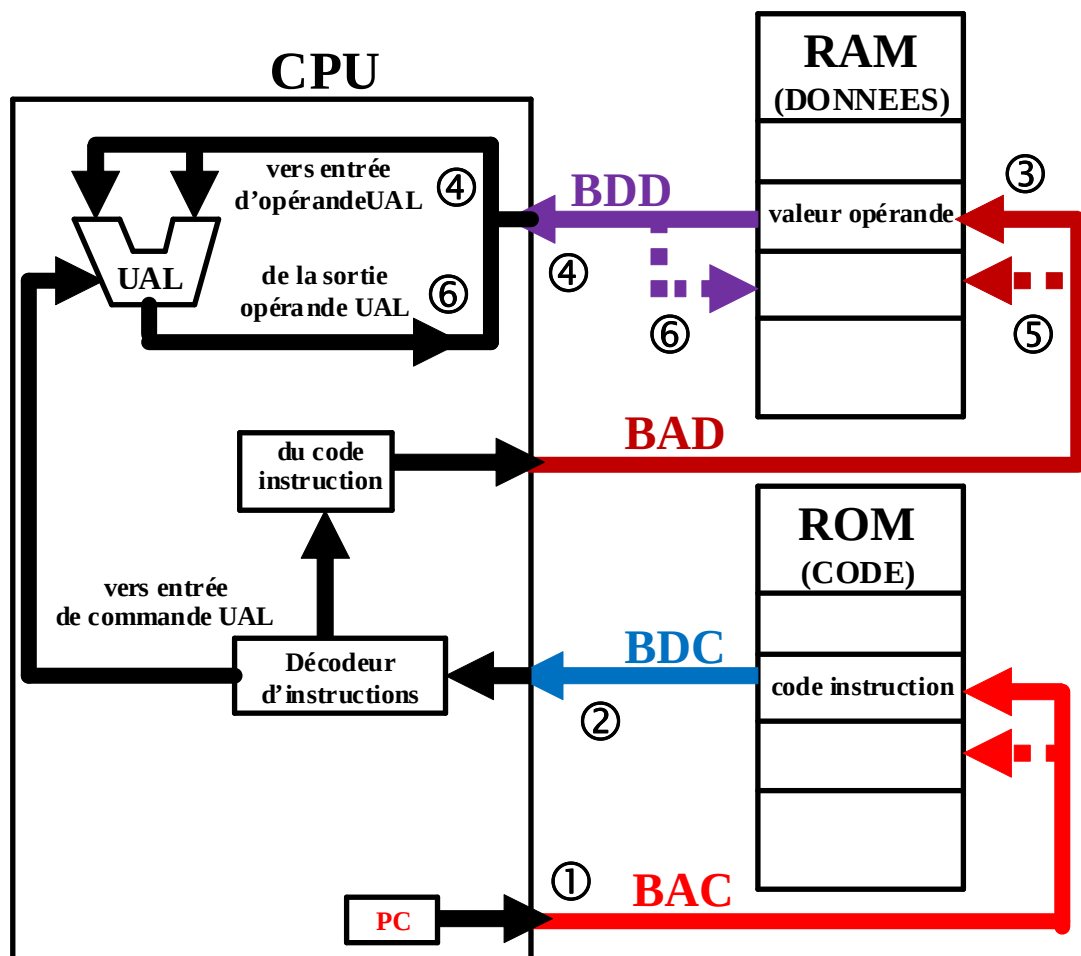


Figure I.21. (page 47)

Quelque soit le micro/processeur/contrôleur,
il existe un registre particulier appelé le

PC : Program Counter

Le PC contient une adresse (c'est un pointeur) :
cette adresse est celle de la case mémoire contenant
le code de la prochaine instruction à exécuter.

Il pointe donc vers la mémoire de code.

Programme = suite d'instructions rangées en mémoire de code
dans des cases mémoires contiguës.

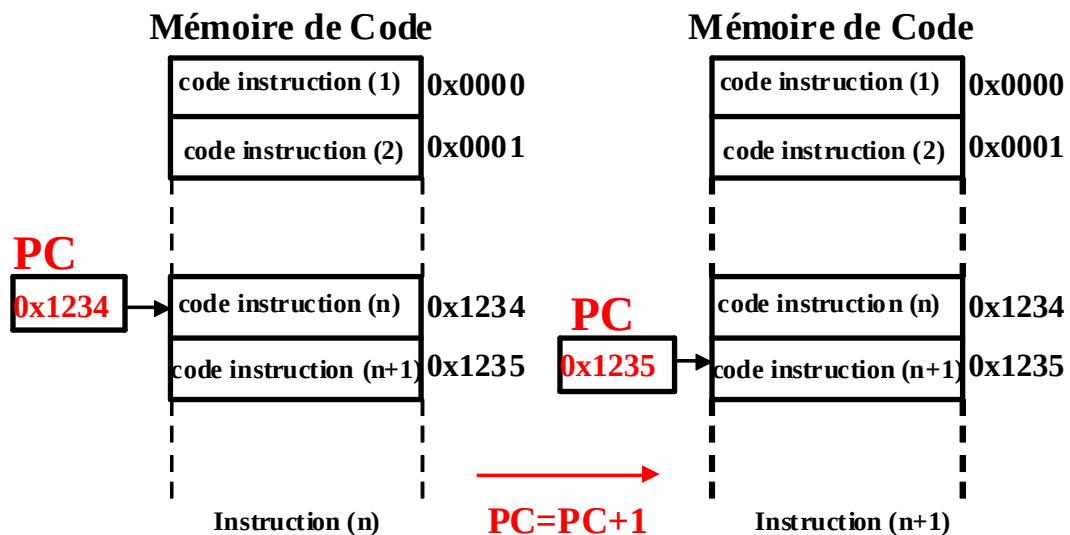


Figure I.22. (page 48)

Le PC joue ainsi le rôle
de séquenceur (ou numérateur) d'instructions.

Séquencement des instructions ≡ « moteur » à **4 temps (étapes)**

Pour les explications, suivre la chronologie :

①, ②, ③ ... de la figure I.21. (page 47)

Etape 1 : INSTRUCTION FETCH (accès ROM)

Recherche de l'instruction (n) en mémoire de code : ① et ②

Etape 2 : INSTRUCTION DECODE (dans CPU et accès RAM)

Décodage de l'instruction (n) dans la CPU :

- différencier les instructions entre elles
- récupération opérande **source** : ③ et ④

Etape 3 : INSTRUCTION EXECUTE 1 (dans CPU)

Exécution de l'instruction dans l'UAL

Etape 4 : INSTRUCTION EXECUTE 2 (accès RAM)

Dans certains cas, rangement résultat dans l'opérande **destination** :
⑤ et ⑥

Résumé du cours PIC : chapitre I

D'où les chronogrammes :

Remarque : il faut 4 « coups d'horloge » (Q1 à Q4) pour un accès mémoire

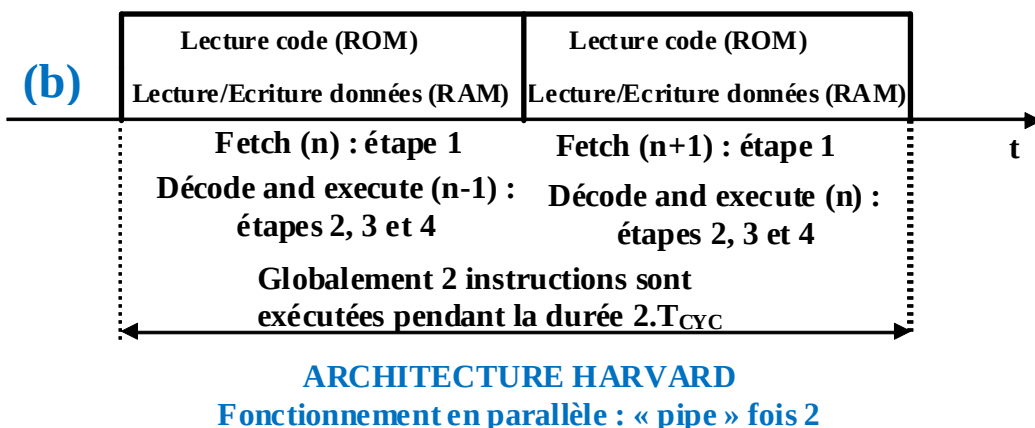
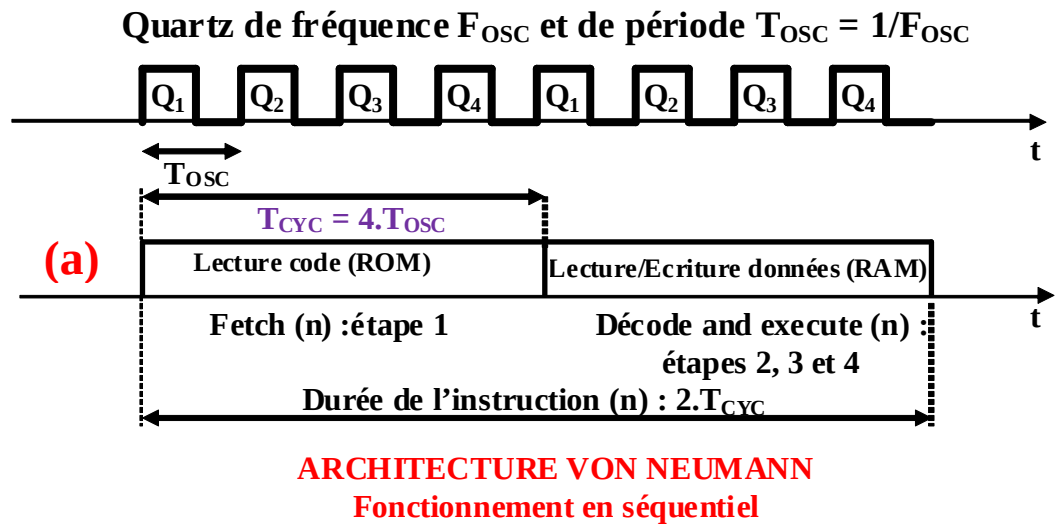


Figure I.23. (page 50)

EXAMPLE 3-1: INSTRUCTION PIPELINE FLOW

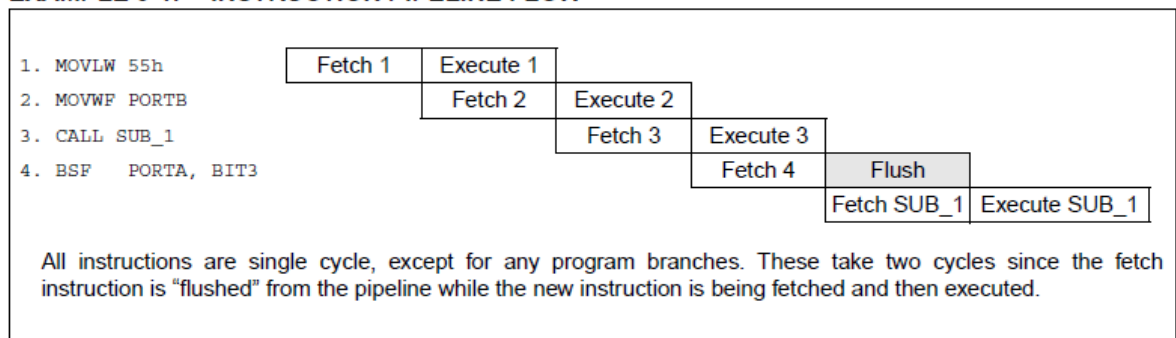


Figure I.24. (page 51)

Vitesse de traitement

Au final, on retiendra :

Pour les PIC, **chaque instruction dure** :

$$T_{CYC} = 4.T_{OSC} = \frac{4}{F_{OSC}}$$

- soit **1 μ s** pour un quartz à **$F_{OSC} = 4$ MHz**
- soit **200 ns** pour un quartz à **$F_{OSC} = 20$ MHz**

(sauf pour les ruptures de séquence qui durent :

$$2.T_{CYC} = 8.T_{OSC} = \frac{8}{F_{OSC}})$$

Ce qui donne une vitesse de traitement de :

- **1 Mips** pour un quartz à **$F_{OSC} = 4$ MHz**
- **5 Mips** pour un quartz à **$F_{OSC} = 20$ MHz.**

Grâce au pipe « fois 2 » sur une architecture HARVARD,
le séquençement des instructions fonctionne comme
« un moteur à quatre temps dont deux temps sont imbriqués »

Résumé du cours PIC : chapitre II

En TP, nous utiliserons le PIC :

16 F 877A - 4

FAMILLE PIC CHEZ MICROCHIP

Type de **ROM implantée**

R = **C** : ROM OTP

R = **F** : FLASH EEPROM

En lien avec le **PIC**

- taille **ROM programme**
- nombre de **broches d'E/S**
- type d'**interfaces...**

n R p - q

Type de **famille**

n = **10** ou **12** : **bas** de gamme

n = **16** : **milieu** de gamme

n = **18** : **haut** de gamme

Fréquence max du quartz système
(**4 Mhz, 20 MHz ...**)

Brochage du PIC 16 F 877A

40-Pin PDIP

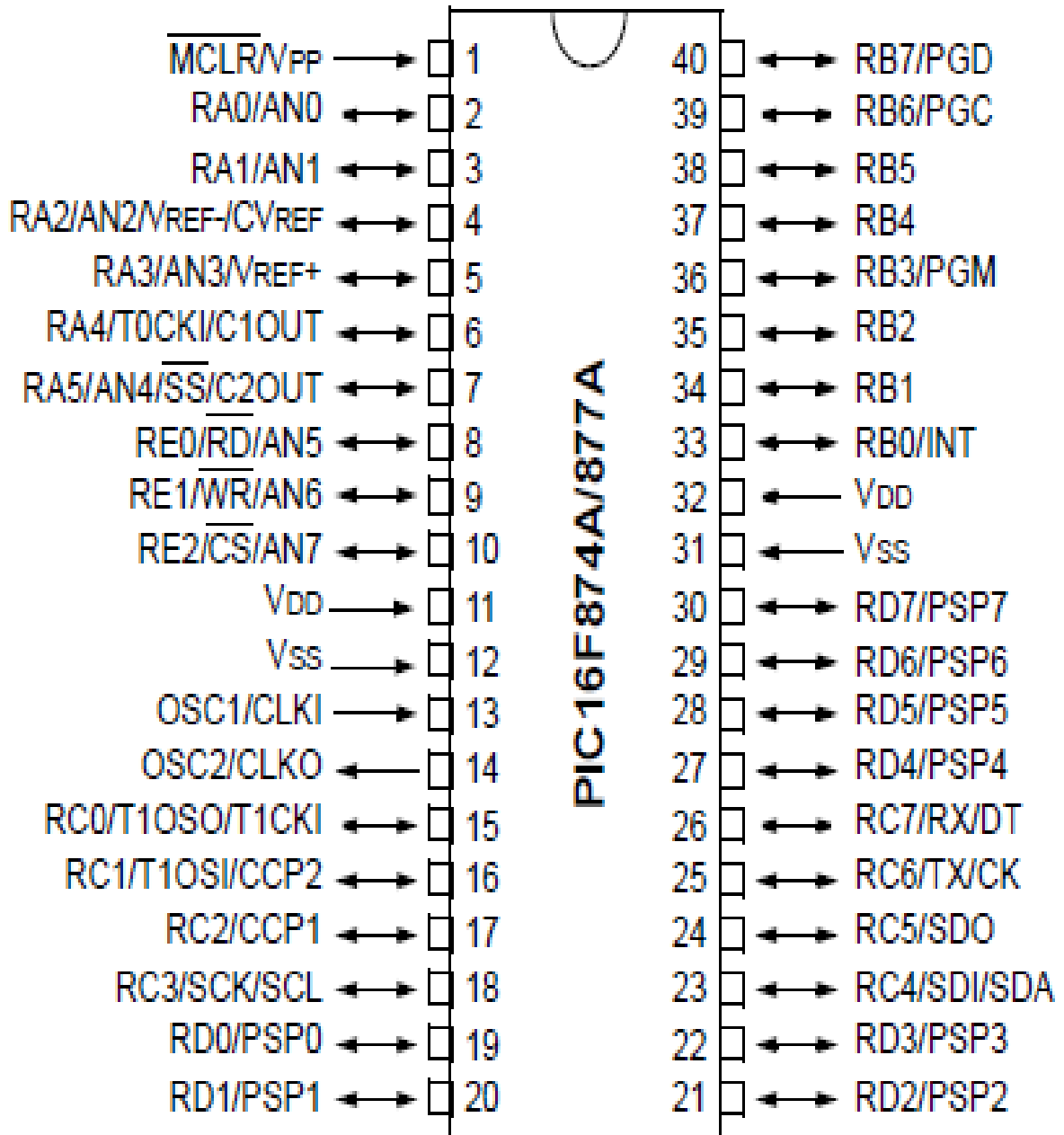


Figure II.1. (page 58)

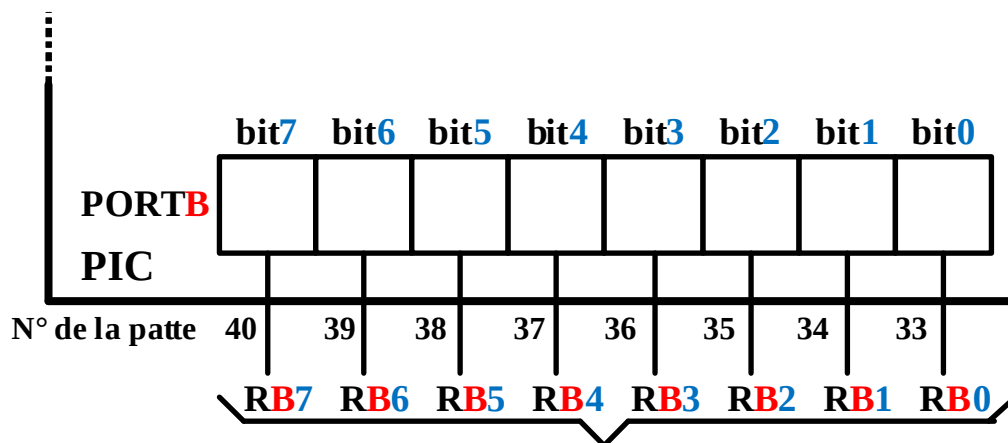
La plupart des broches du PIC sont « **multifonctionnelles** »

Repérage des broches des PORT d'E/S

5 PORTS d'Entrée/Sortie logiques (dits aussi ports **parallèles**)
(ou **Input/Output PORT** ou **IO PORT**).

PORTA, **PORTB**, **PORTC**, **PORTD** et **PORTE**

Exemple pour le **PORTB** de taille **8 bits**.



Broches externes du PIC

Figure II.2. (page 59)

Un port se comporte donc

comme une **case mémoire** (en lecture/écriture)
(son registre de port, comme par exemple **PORTB** ci-dessus)

possédant un **accès sur l'extérieur**
(comme par exemple les broches **RB7** à **RB0** ci-dessus)

Architecture interne en résumé :

avec repérage des **blocs fonctionnels internes** par les points ① à ⑫

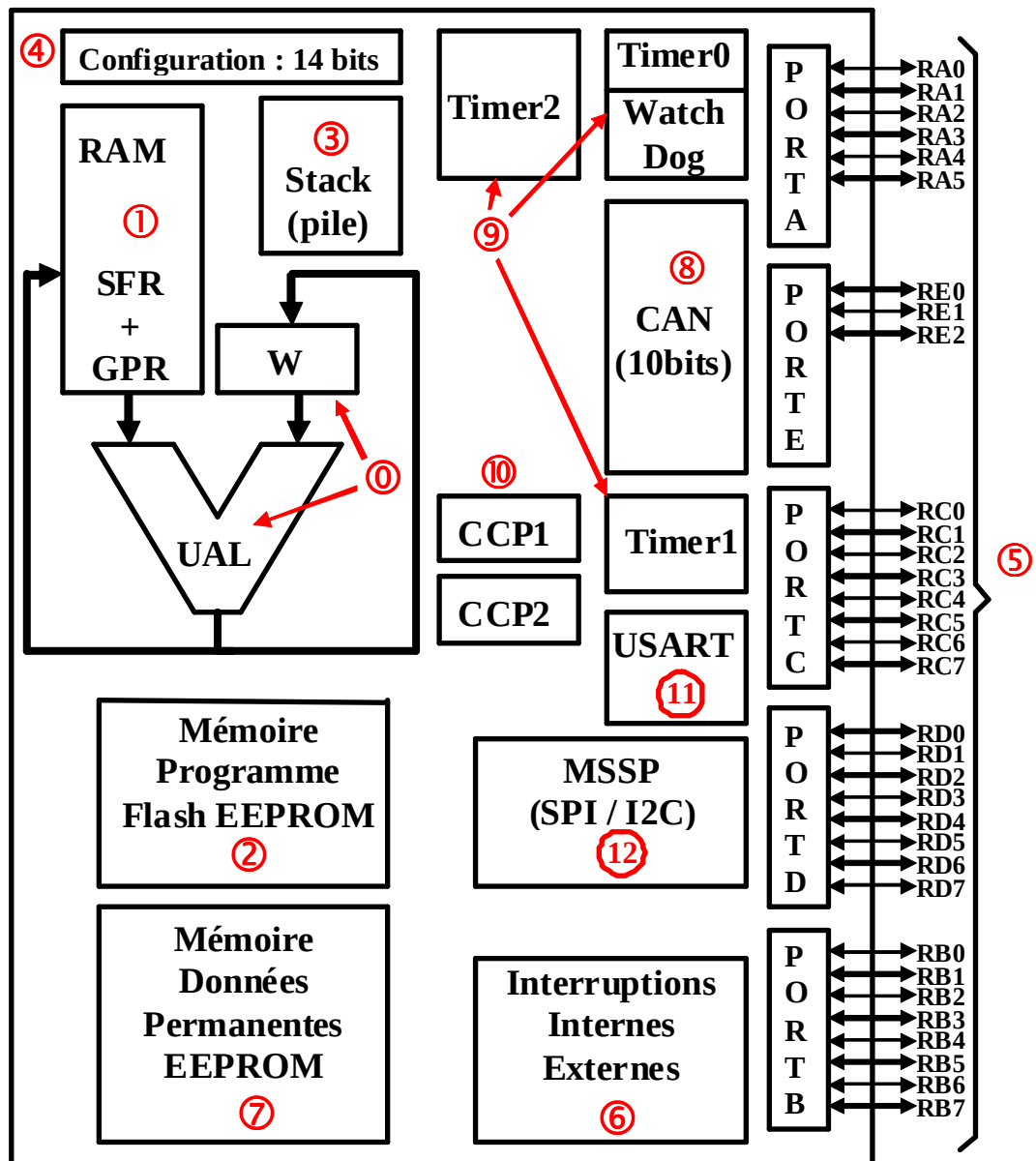


Figure II.4. (page 61)

Principales caractéristiques

- ① : **UAL** et accumulateur **W** :
W est un registre général de travail (**W**ork) de taille **8 bits**.
- ① : **RAM interne** de **512** cases de taille **1 octet** chacune :
chaque case est appelée aussi **register file** (par MICROCHIP).
On y distingue au maximum :
 - **96 SFR** ou **S**pecial **F**unction **R**egisters :
registres internes (fonctionnement et interfaces)
 - **416 GPR** ou **G**eneral **P**urpose **R**egisters :
pour les variables des programmes utilisateur.
- ② : **FLASH EEPROM interne**
de **8K** cases de taille **14 bits** chacune :
pour le programme applicatif (les instructions).
- ③ : une **pile (STACK)** possédant **8** positions mémoire
de taille **13 bits** chacune :
au max **8 niveaux d'imbrication** de fonctions/interruption.
- ④ : **registre de configuration** de taille **14 bits** :
pour fixer des règles de sécurité et de fonctionnement pour le
programme implanté.
- ⑤ : **33 lignes d'entrée/sorties bidirectionnelles**
réparties sur 5 ports parallèles :
PORTA, **PORTB**, **PORTC**, **PORTD** et **PORTE**.

Principales caractéristiques (suite)

- ⑥ : un module de **gestion des interruptions** intérieures et extérieures :
au total **14 sources d'interruption** sont implantables.
- ⑦ : **FLASH EEPROM interne** de **256** octets :
pour les données (semi) permanentes du programme.
- ⑧ : **CAN** (Convertisseur Analogique/Numérique) :
avec **10 bits** de précision et **8** canaux de conversion.
- ⑨ : **trois timers/compteurs indépendants** **TIMER0**, **TIMER1** et **TIMER2** :
pour le comptage du temps et/ou des événements extérieurs.
- ⑩ : **deux modules CCP1 et CCP2** :
pour dater des événements et/ou générer des signaux **PWM**.
- ⑪ : **USART** :
(Universal Asynchronous Synchronous Receiver Transmitter)
pour les liaisons série.
- ⑫ : un bloc **MSSP** (Master Slave Serial Port) :
pour gérer les liaisons série **synchrones** de type **SPI** ou **I²C**.

① UAL et accumulateur W

L'UAL exécute les **instructions élémentaires** situées en **ROM**.

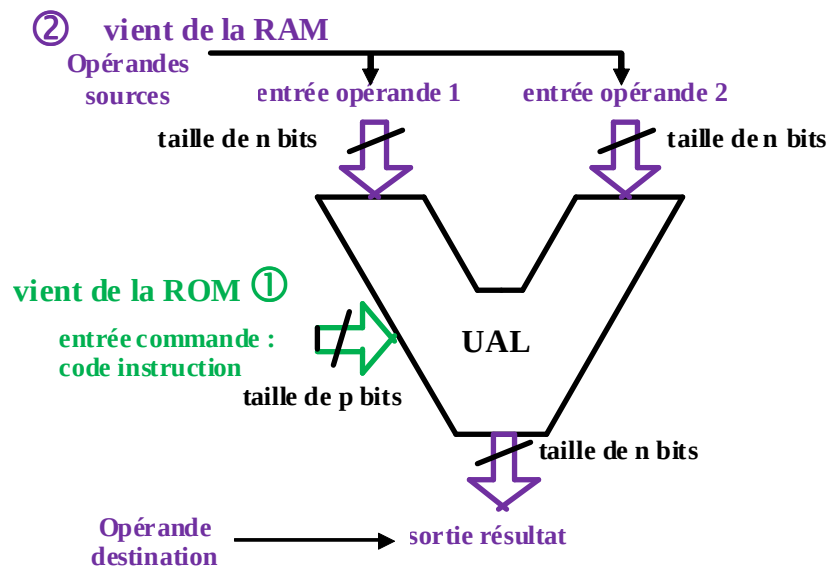


Figure I.20. (page 46)

Ce que fait l'instruction :

- traitement **arithmétique** : ADD, SUB, INC, DEC,
- traitement **logique** : AND, IOR, XOR, COM, RR, RL,
- **déplacement** mémoire : MOV,
- **rupture de séquence** : GOTO, CALL...

Stockée
en **ROM**
de programme

Instruction = **Mnémonique** + **opérande(s)**

Adresses des données concernées
par le traitement du **mnémonique**

- opérande **source**
- opérande **destination** (éventuellement)

Résumé du cours PIC : chapitre III

Un programme écrit en **assembleur** sera donc une suite d'instructions situées dans **des cases contiguës de la ROM de programme** (cf. figure I.22. page 48)

Codage machine d'une instruction en ROM

Rappel

chaque case ROM a pour taille **14 bits**.

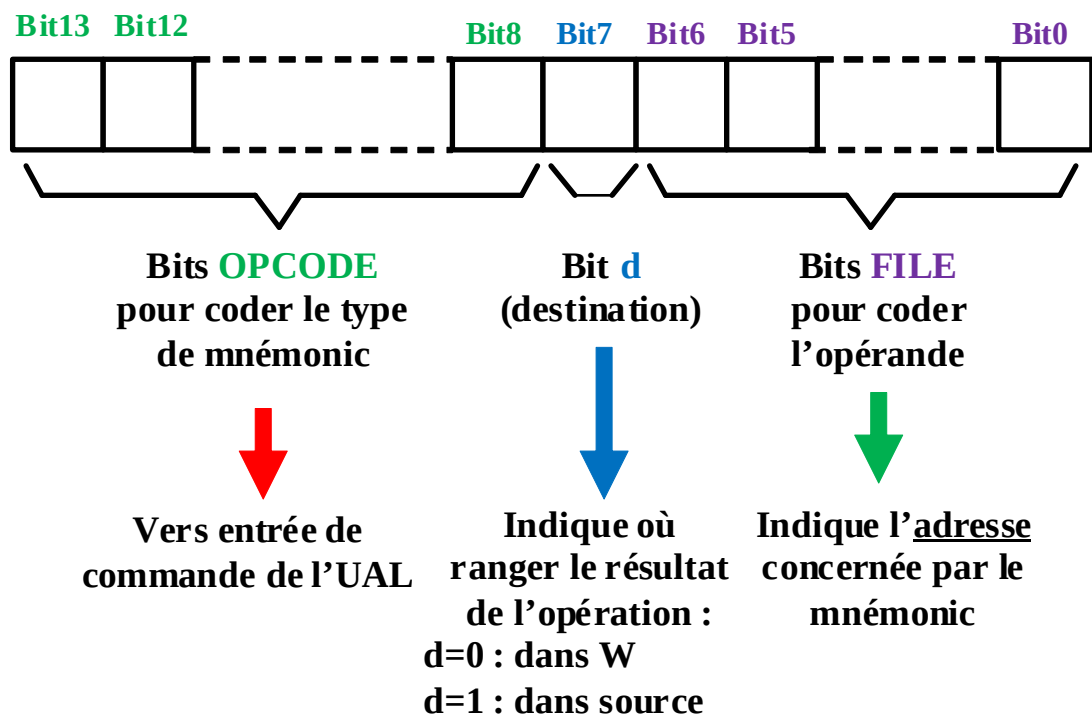


Figure III.1. (page 67)

① La mémoire RAM interne (512 octets)

SFR : registres « système » et registres de « contrôle de périphériques »

GPR : registres « utilisateur » : pour les variables du programme.

Pour accéder à ces **512** registres ($=2^9$), il faut **9 bits d'adresses...**

Or, l'adresse de l'opérande est codée sur **7 bits ...**
(cf. champ **Bits FILE** de la figure précédente).

Conclusion : il manque 2 bits !!!

D'où la solution obtenue par le découpage suivant de la **RAM** :

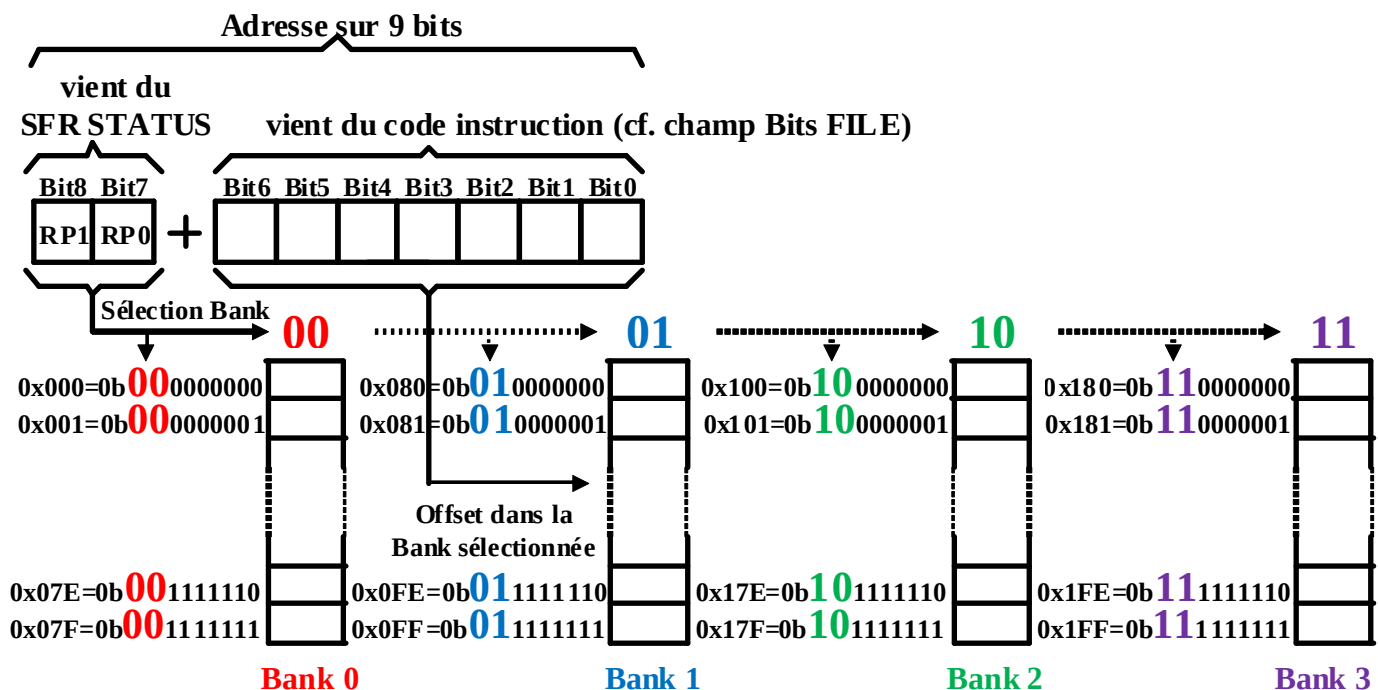


Figure III.2. (page 68)

Mapping de la mémoire RAM interne

Les 2 bits, appelés **RP1** et **RP0** proviennent du registre **SFR STATUS**.

Ils seront rajoutés par le **PIC** pour former une adresse de **9 bits** permettant d'accéder à l'opérande en **mode d'adressage direct**.

Les **512** octets de la RAM interne seront donc divisés
en **4 banks** (terminologie de **MICROCHIP**)
de **128 octets** chacune :

Bank 0 : de 0x000 à 0x07F (RP1RP0=00)

Bank 1 : de 0x080 à 0x0FF (RP1RP0=01)

Bank 2 : de 0x100 à 0x17F (RP1RP0=10)

Bank 3 : de 0x180 à 0x1FF (RP1RP0=11)

Fonctionnement de l'adressage direct

- **RP1-RP0** : choix de la bank,
 - les **7 bits** issus du code instruction
 - = offset dans la bank sélectionnée
 - = décalage par rapport à l'adresse de base (du début) de la bank sélectionnée
 - = choix d'une case parmi les 128 cases contenues dans la bank sélectionnée
- par **RP1-RP0**.

💣* Conséquence importante

Pour accéder à une case mémoire de la RAM,
en utilisant **l'adressage direct en assembleur**,
il faudra donc auparavant penser à positionner correctement
les bits **RP1-RP0** en fonction de la bank où se trouve cette case !!!
Par contre **en C, pas de précaution particulière** à prendre !!!

Résumé du cours PIC : chapitre III

Mapping de la mémoire RAM interne (suite)

File Address	File Address	File Address	File Address
Indirect addr. ^(*) 00h	Indirect addr. ^(*) 80h	Indirect addr. ^(*) 100h	Indirect addr. ^(*) 180h
TMR0 01h	OPTION_REG 81h	TMR0 101h	OPTION_REG 181h
PCL 02h	PCL 82h	PCL 102h	PCL 182h
STATUS 03h	STATUS 83h	STATUS 103h	STATUS 183h
FSR 04h	FSR 84h	FSR 104h	FSR 184h
PORTA 05h	TRISA 85h	/ 105h	/ 185h
PORTB 06h	TRISB 86h	PORTB 106h	TRISB 186h
PORTC 07h	TRISC 87h	/ 107h	/ 187h
PORTD ⁽¹⁾ 08h	TRISD ⁽¹⁾ 88h	/ 108h	/ 188h
PORTE ⁽¹⁾ 09h	TRISE ⁽¹⁾ 89h	/ 109h	/ 189h
PCLATH 0Ah	PCLATH 8Ah	PCLATH 10Ah	PCLATH 18Ah
INTCON 0Bh	INTCON 8Bh	INTCON 10Bh	INTCON 18Bh
PIR1 0Ch	PIE1 8Ch	EEDATA 10Ch	EECON1 18Ch
PIR2 0Dh	PIE2 8Dh	EEADR 10Dh	EECON2 18Dh
TMR1L 0Eh	PCON 8Eh	EEDATH 10Eh	Reserved ^(*) 18Eh
TMR1H 0Fh	/ 8Fh	EEADRH 10Fh	Reserved ^(*) 18Fh
T1CON 10h	/ 90h	/ 110h	/ 190h
TMR2 11h	SSPCON2 91h	/ 111h	/ 191h
T2CON 12h	PR2 92h	/ 112h	/ 192h
SSPBUF 13h	SSPADDD 93h	/ 113h	/ 193h
SSPCON 14h	SSPSTAT 94h	/ 114h	/ 194h
CCPR1L 15h	/ 95h	/ 115h	/ 195h
CCPR1H 16h	/ 96h	/ 116h	/ 196h
CCP1CON 17h	/ 97h	General Purpose Register 16 Bytes 117h	General Purpose Register 16 Bytes 197h
RCSTA 18h	TXSTA 98h	/ 118h	/ 198h
TXREG 19h	SPBRG 99h	/ 119h	/ 199h
RCREG 1Ah	/ 9Ah	/ 11Ah	/ 19Ah
CCPR2L 1Bh	/ 9Bh	/ 11Bh	/ 19Bh
CCPR2H 1Ch	/ 9Ch	/ 11Ch	/ 19Ch
CCP2CON 1Dh	/ 9Dh	/ 11Dh	/ 19Dh
ADRESH 1Eh	ADRESL 9Eh	/ 11Eh	/ 19Eh
ADCON0 1Fh	ADCON1 9Fh	/ 11Fh	/ 19Fh
/ 20h	/ A0h	/ 120h	/ 1A0h
General Purpose Register 96 Bytes	General Purpose Register 80 Bytes	General Purpose Register 80 Bytes	General Purpose Register 80 Bytes
Zone commune 70h	accesses 70h-7Fh	accesses 70h-7Fh	accesses 70h-7Fh
Bank 0 7Fh	Bank 1 FFh	Bank 2 17Fh	Bank 3 1FFh

 Unimplemented data memory locations, read as '0'.
^{*} Not a physical register.

Figure III.3. (page 70)

Résumé du cours PIC : chapitre III

Cinq remarques sur les SFR du mapping RAM :

① Les cases avec un nom à l'intérieur sont des **SFR** :
c'est **le nom de ce registre SFR**.

Exemple : TMR0, PCL, STATUS, FSR, PORTA, PORTB ...

TMR0	01h
PCL	02h
STATUS	03h
FSR	04h
PORTA	05h
PORTB	06h

② Certains sont présents dans plusieurs banks, avec le même offset (principe des adresses fantômes ou **ghosts**)

Exemple : STATUS à l'offset **0x003** de chaque bank
et donc aux **adresses absolues** :

- **0x003** : dans la bank 0,
 - **0x083** : dans la bank 1,
 - **0x103** : dans la bank 2,
 - **0x183** : dans la bank 3.
- Quel est, à votre avis, l'intérêt d'avoir un même SFR présent au même offset dans les 4 banks ?

STATUS	03h	STATUS	83h	STATUS	103h	STATUS	183h
--------	-----	--------	-----	--------	------	--------	------

③ Les zones **grisées et hachurées** ne correspondent à aucun **SFR**.

④ Le registre à l'offset **0x000** de toutes les banks (**Indirect addr**) sert pour l'**adressage indirect** (voir plus loin).

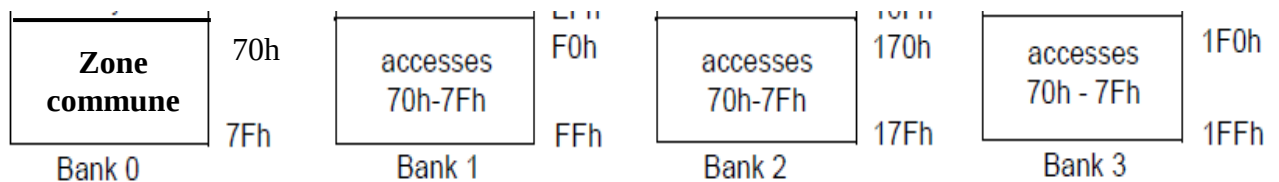
Indirect addr. ^(*)	00h	Indirect addr. ^(*)	80h	Indirect addr. ^(*)	100h	Indirect addr. ^(*)	180h
-------------------------------	-----	-------------------------------	-----	-------------------------------	------	-------------------------------	------

⑤ Au final, et compte tenu des 4 remarques précédentes, il y a :

54 SFR différents.

Trois remarques sur les GPR du mapping RAM

- ① Les cases sans nom à l'intérieur sont des **GPR**
(cases disponibles pour stocker les variables de votre programme)
- ② Les **16 derniers octets** de **chaque bank** sont une **zone commune** :



Cela signifie que les plages d'adresses :

0x0F0-0x0FF (bank 1),
0x170-0x17F (bank 2),
0x1F0-0x1FF (bank 3),

correspondent à la même plage d'adresses que :

0x070-0x07F (bank 0).

Conséquence :

Ecrire (ou lire) par exemple à l'adresse :
0x0F5 (ou 0x175 ou 0x1F5)
 revient à écrire (ou lire) à l'adresse **0x075**.

- ③ Au final, et compte tenu des 2 remarques précédentes, il y a :
368 GPR différents.

Résumé du cours PIC : chapitre III

Le registre STATUS

(ou registre d'état)

On trouvera dans **tous** les micro/processeurs/contrôleurs un **registre d'état** (avec bien sûr un autre format !!!)

Il sert à transmettre des informations sur l'**état** de la CPU **après** l'exécution de certaines instructions, informations utiles qui peuvent être ensuite **testées** par l'**instruction suivante** (ce qu'on appelle une rupture conditionnelle de séquence) pour assurer le bon déroulement du programme.

FORMAT DU REGISTRE D'ETAT STATUS

STATUS REGISTER (situé aux adresses 0x003, 0x083, 0x103, 0x183)
(cf.figure III.3. page 70)

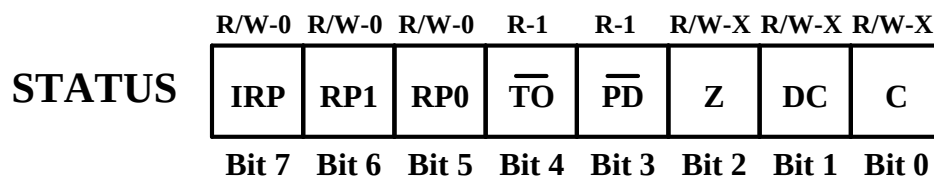


Figure III.4. (page 73)

FUSE	02h	FUSE	02h	FUSE	102h	FUSE	102h
STATUS	03h	STATUS	83h	STATUS	103h	STATUS	183h
FSR	04h	FSR	84h	FSR	104h	FSR	184h
PORTA	05h	TRISA	85h		105h		185h

Bits $\overline{\text{TO}}$ (comme **T**ime **O**ut) et $\overline{\text{PD}}$ (comme **P**ower **D**own) liés à des fonctionnements particuliers (basse consommation notamment).

Bits **RP1** et **RP0** (déjà vus) pour **l'adressage direct**.
(cf. $i = 5$ du C où i est déclaré comme un int)

Bit **IRP** pour **l'adressage indirect**.
(cf. $*p = 5$ du C où p est déclaré comme un int *)

Résumé du cours PIC : chapitre III

Le registre STATUS (suite)

STATUS REGISTER (situé aux adresses 0x003, 0x083, 0x103, 0x183)
(page 18 du datasheet)

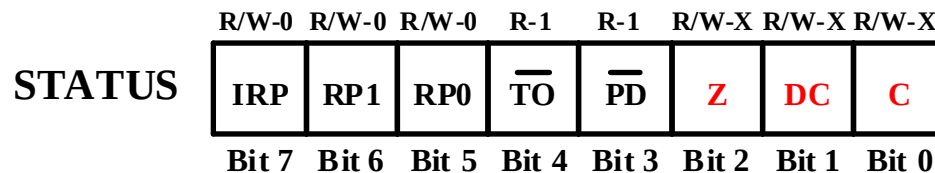


Figure III.4. (page 73)

Les indicateurs d'états (ou FLAGS) : bits Z, DC et C

Flag Z comme Zero

Certaines instructions peuvent fournir un résultat **nul**. Par exemple :

- si on incrémente le registre **W** alors qu'il contient **0xFF**,
- si on soustrait deux opérandes de **valeur égale**,
- si on charge une valeur **nulle** dans un **GPR**...

Dans ces cas là, le bit **Z** sera positionné à **1**.

Par contre, **ces mêmes instructions** peuvent aussi, dans d'autres conditions, donner un résultat **non nul** :

- on incrémente **W** qui contient, avant, **autre chose que 0xFF**,
- on soustrait deux opérandes **différents**,
- la valeur chargée dans un **GPR** est **non nulle**...

Dans ces cas là, le bit **Z** sera positionné à **0**.

Résumé du cours PIC : chapitre III

Flag **C** comme **C**arry (retenue)

Sert pour indiquer la **retenue**. Par exemple :

9^{ème} bit du résultat dans le cas d'une **addition** entre **deux octets**.

Sert aussi pour les **soustractions** : on l'appelle alors **BORROW**

Rappel sur le codage des entiers (ici sur un octet)

		①	②
Hexadécimal	Binaire (naturel)	Non signé (en décimal)	Signé (en décimal)
0x00	0b 0000 0000	0	0
0x01	0b 0000 0001	1	+1
0x02	0b 0000 0010	2	+2
....			
0x7E	0b 0111 1110	126	+126
0x7F	0b 0111 1111	127	+127
0x80	0b 1000 0000	128	-128
0x81	0b 1000 0001	129	-127
0x82	0b 1000 0010	130	-126
....			
0xFE	0b 1111 1110	254	-2
0xFF	0b 1111 1111	255	-1

Figure III.5. (page 75)

Colonne ① : entiers non signés $\Rightarrow$ binaire naturel

- sur **1 octet** de : **0** à $2^8 - 1$ soit de **0** à **255**,
- sur **2 octets** de : **0** à $2^{16} - 1$ soit de **0** à **65 535**,
- sur **4 octets** de : **0** à $2^{32} - 1$ soit de **0** à **4 294 967 295**.

Codage utilisé par le C sur des déclarations du type :

unsigned	char // taille 8 bits
unsigned (short)	int // taille 16 bits
unsigned long	int // taille 32 bits

Colonne ② : entiers signés $\Rightarrow$ complément à deux

- sur **1 octet** de : **-2^7** à **$2^7 - 1$** soit de **-128** à **$+127$** ,
- sur **2 octets** de : **-2^{15}** à **$2^{15} - 1$** soit de **$-32 768$** à **$+32 767$**
- sur **4 octets** de : **-2^{31}** à **$2^{31} - 1$** (à calculer ...)

Codage utilisé par le C sur des déclarations du type :

signed	char // taille 8 bits
signed (short)	int // taille 16 bits
signed long	int // taille 32 bits

Le bit **MSB** (bit de signe) prend un sens particulier :

- **MSB = 0** pour tous les entiers **positifs ou nul**,
- **MSB = 1** pour tous les entiers **négatifs**.

Exemple

Codage de **(-5)** sur un **octet** en **binaire signé**.

Valeur absolue de -5 :	0b 0000 0101	
Inversion de tous les bits :	0b 1111 1010	(complément à 1)
Ajout de 1 :	+	0b 0000 0001
Soit au final :	=	0b 1111 1011 = 0xFB

Donc **(-5)** sera codé **0xFB** en binaire signé.

Remarque

Se rappeler que l'entier **(-1)** sera codé :

0xFF	:	sur 1 octet ,
0xFF FF		sur 2 octets ,
0xFF FF FF FF		sur 4 octets...

Résumé du cours PIC : chapitre III

Comment fonctionne alors une soustraction ?

Et bien comme une addition !!!

$$A - B = A + (-B)$$

A	B	A-B	Codage de -B	A+(-B)	Z	C
7	5	2	0b 1111 1011	0b 0000 0111 + 0b 1111 1011 = 1 0b 0000 0010 (+2 OK)	0	1
7	7	0	0b 1111 1001	0b 0000 0111 + 0b 1111 1001 = 1 0b 0000 0000 (0 OK)	1	1
7	9	-2	0b 1111 0111	0b 0000 0111 + 0b 1111 0111 = 0 0b 1111 1110 (-2 OK)	0	0

Figure III.6. (page 78)

Au final :

- si **Z = 1** alors : **A = B**,
- si **C = 1** alors : **A ≥ B**,
- si **C = 0** alors : **A < B**.

Résumé du cours PIC : chapitre III

Ce qui permettra d'implanter par exemple les tests conditionnels que l'on trouve dans les langages de haut niveau comme :

if (A == B) {.....} else {.....}; // par test à **1** du flag **Z**

if (A >= B) {.....} else {.....}; // par test à **1** du flag **C**

if (A < B) {.....} else {.....}; // par test à **0** du flag **C**

....

Flag DC comme **D**igit **C**arry

Sert pour indiquer la retenue entre le **bit 3** et le **bit 4** :
utile en arithmétique **DCB**.
(voir exemple polycopié page 79)

On retiendra que :

- si certaines instructions modifient les flags
en les positionnant à **1** ou à **0** suivant le résultat obtenu
(faire espèce de « nano-photographie » sur 3 bits de l'état CPU),
- d'autres, immédiatement après, devront les tester
(exploiter la « nano-photographie » de l'instruction précédente)
pour prendre en compte ces positionnements.
(principe d'une rupture conditionnelle synchrone de séquence)

Résumé du cours PIC : chapitre III

② La mémoire ROM interne (8K mots de 14 bits chacun)

Elle contient le **code** (les **instructions**) du programme.

Mémoire **FLASH EEPROM** paginée en **4** pages de **2K mots**

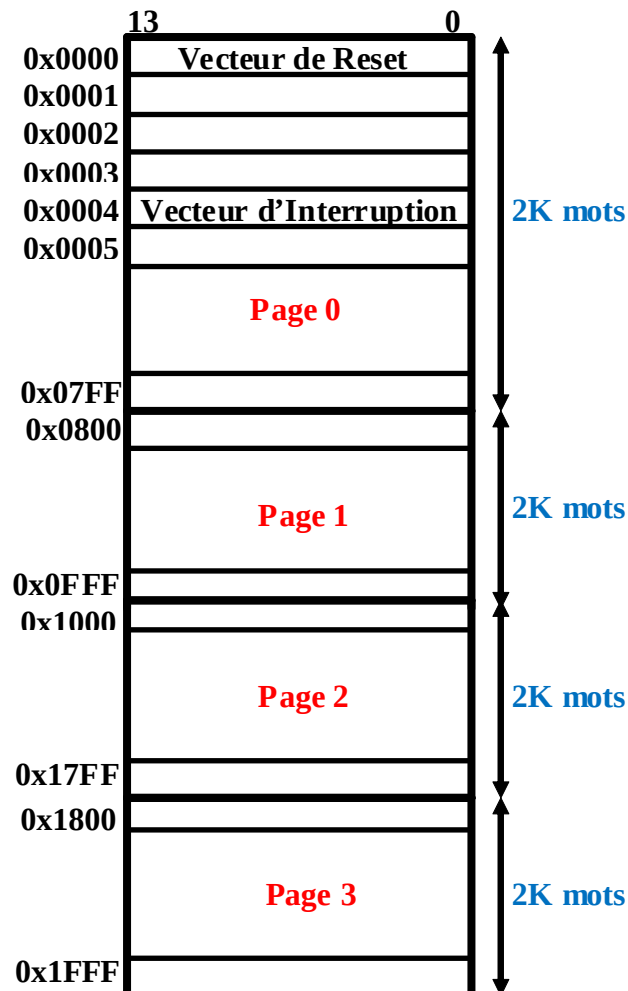


Figure III.9. (page 85)

On rappelle que chaque case contient une instruction du programme (codée en langage machine).

Or, pour accéder à ces **8K mots** ($=8192 = 2^{13}$)
il faut **13 lignes d'adresses**.

Résumé du cours PIC : chapitre III

Registre PC (compléments)

Rappel

**Il pointe sur la prochaine instruction à exécuter : il contient son adresse.
c'est un registre pointeur vers la mémoire de code.**

Sa taille doit donc être de **13** bits.

Il est constitué par la **concaténation** de deux registres **SFR** :

- **PCL** : à l'offset **0x02** de chaque bank,

TMR0	01h	OPTION_REG	81h	TMR0	101h	OPTION_REG	181h
PCL	02h	PCL	82h	PCL	102h	PCL	182h
STATUS	03h	STATUS	83h	STATUS	103h	STATUS	183h

- **PCLATH** : à l'offset **0x0A** de chaque bank.

PORTE ⁽¹⁾	09h	TRISE ⁽¹⁾	89h		109h		189h
PCLATH	0Ah	PCLATH	8Ah	PCLATH	10Ah	PCLATH	18Ah
INTCON	0Bh	INTCON	8Bh	INTCON	10Bh	INTCON	18Bh

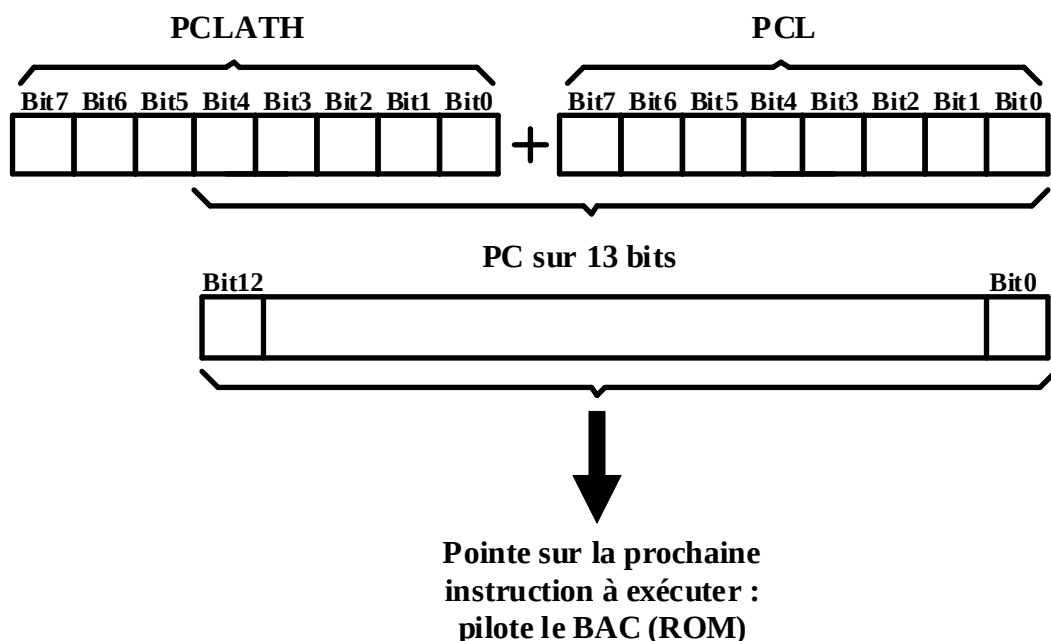


Figure III.12. (page 87)

Résumé du cours PIC : chapitre III

Gestion du registre PC

Structure d'un programme « source » écrit en assembleur

Trois différents champs :

Champ des adresses (labels, étiquettes)	Champ des instructions (mémonics + opérandes mnémonics)	Champ des commentaires (commençant par ;)
--	--	--

debut	Instruction 1	; commentaire 1
	Instruction 2	; commentaire 2
	Instruction 3	; commentaire 3
		

Figure III.10. (page 86)

Bien respecter ces alignements en colonnes pour augmenter la lisibilité et la compréhension de vos programmes !!!

Ce qui correspond à l'implantation suivante en ROM de programme:
(on a supposé ici que le programme commence à l'adresse **0x0800**)

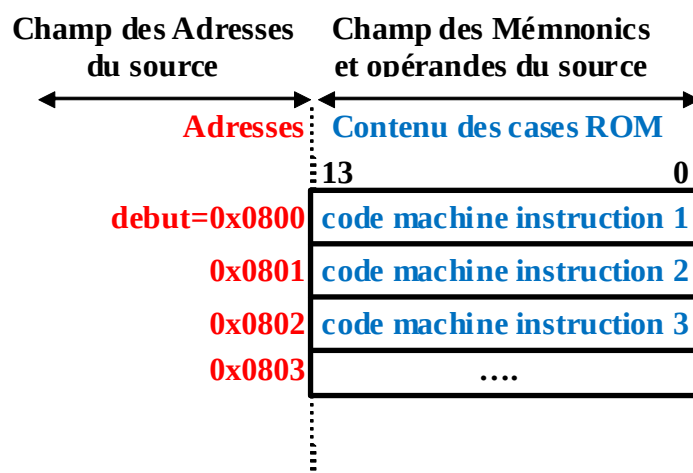


Figure III.11. (page 86)

Le **code machine de l'instruction 1** se trouve à l'adresse **0x0800**,
le **code machine de l'instruction 2** à l'adresse suivante **0x0801**,
le **code machine de l'instruction 3** à l'adresse suivante **0x0802**,
etc. ...

Gestion du registre PC (suite)

Le cas précédent est un cas de **déroulement séquentiel** d'un programme.
(l'instruction suivante à exécuter se trouve dans la case mémoire suivante c'est-à-dire qu'à chaque instruction le **registre PC est incrémenté de 1**)

Ce fonctionnement séquentiel (« de A à Z ») ne permet pas d'implanter des structures algorithmiques comme les boucles, les tests conditionnels que l'on trouve dans les langages de haut niveau.

Cas des ruptures synchrones de séquence

Deux exemples :

① instruction **GOTO** de format :

GOTO **label** ; on saute à l'adresse label

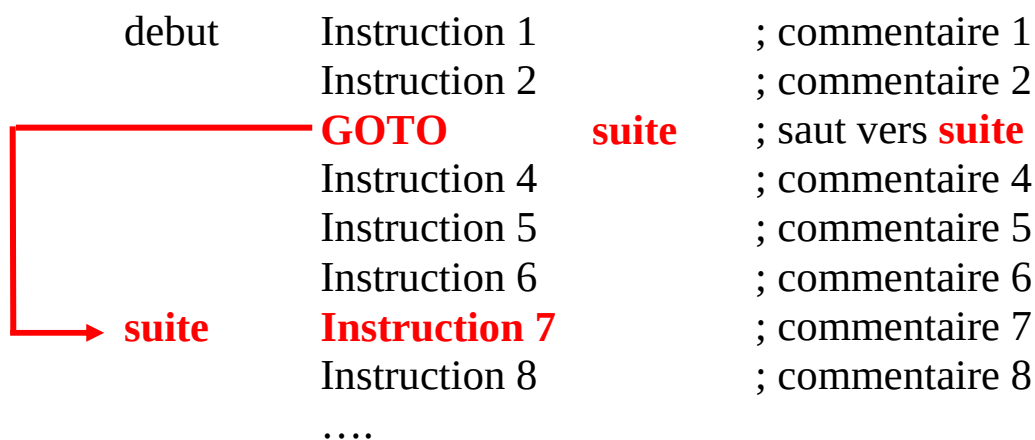


Figure III.13. (page 88)

Remarque

Il s'agit ici d'un **saut** (ou branchement) **inconditionnel** (toujours).

On verra qu'il existe aussi des instructions de **saut** appelées **saut conditionnel**

qui ne se réalisent que si une **certaine condition est vérifiée**.
(par exemple sur la valeur d'un des flags **Z** ou **C**).

Résumé du cours PIC : chapitre III

② instruction **CALL** de format :

CALL nom_sous_programme

Un sous-programme (**module** en assembleur ou **fonction** en C) est un bout de programme qui pourra être exécuté plusieurs fois pour effectuer des tâches répétitives (à chaque fois qu'on a en besoin) :

Par exemple (et liste non exhaustive...) :

- celle de recevoir (et donc de récupérer) un caractère arrivant sur une liaison et qui sera répétée à chaque nouveau caractère reçu,
- celle pour allumer un pixel en (X,Y) sur un écran graphique et qui sera répétée plusieurs fois (en changeant les cordonnées du pixel) pour tracer une droite, un cercle ou toute autre forme géométrique,
- celle qui consiste à faire une conversion analogique/numérique et qui sera répétée à chaque période d'échantillonnage ...

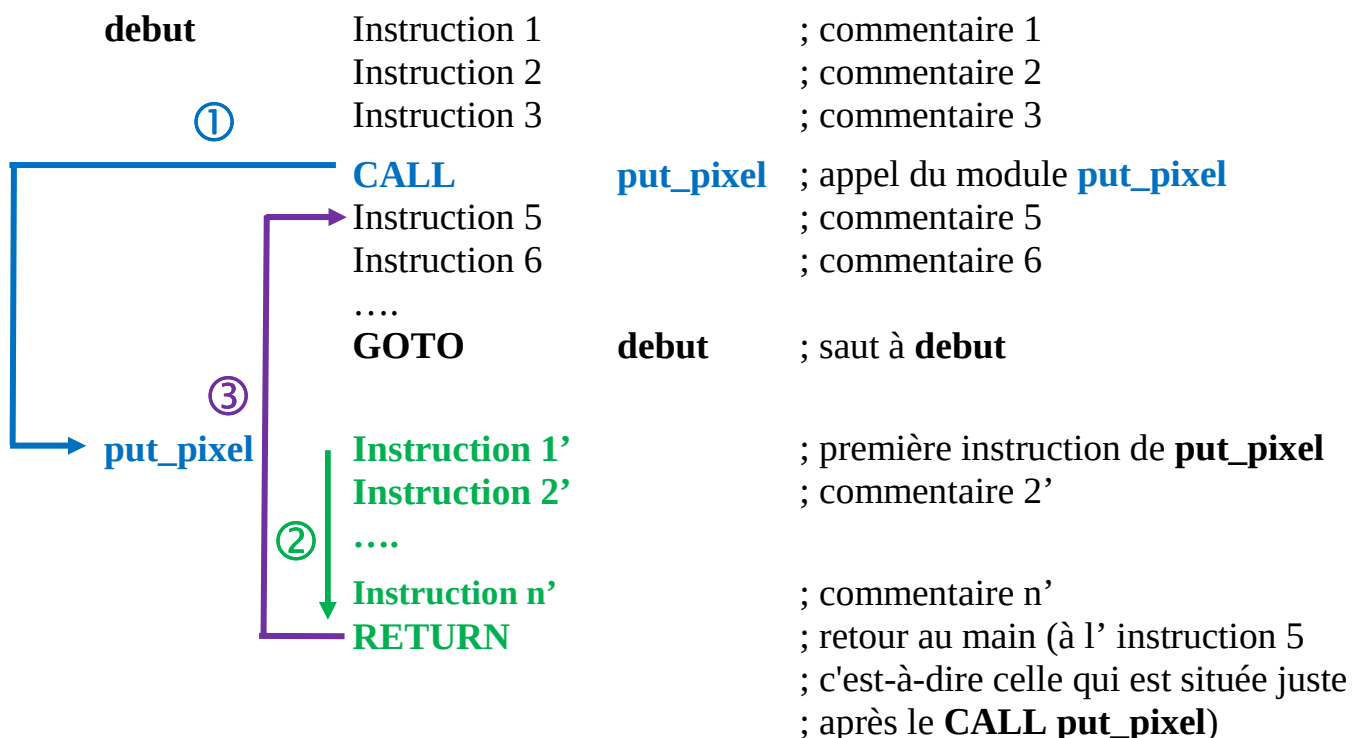


Figure III.14. (page 91)

Résumé du cours PIC : chapitre III

Reprenons l'exemple précédent, en regardant son implantation en mémoire EEPROM de programme :

les adresses utilisées n'ont rien de général, mais ont été choisies juste pour concrétiser cet exemple...

adresses	contenu des cases EEPROM (14 bits)
0x0800	Code machine de l'instruction 1
0x0801	Code machine de l'instruction 2
0x0802	Code machine de l'instruction 3
0x0803 ①	Code machine du CALL 0x0900
0x0804 →	Code machine de l'instruction 5
0x0805	Code machine de l'instruction 6
	
0x0850	Code machine du GOTO 0x0800
③ → 0x0900	Code machine de l'instruction 1'
0x0901	Code machine de l'instruction 2'
	
0x092F	Code machine de l'instruction n'
0x0930	Code machine du RETURN

①

②

③

Résumé du cours PIC : chapitre III

La question qui se pose pour une instruction **CALL** est la suivante :

Dans la mesure où le PC est modifié lors de l'appel du module

(en ①, **phase du ticket « aller »**, il prend en effet la valeur **0x0900** pour sauter à la première instruction du module : ici instruction 1')

comment le programme peut-il revenir à l'adresse **0x0804 ?**

(en ③, **phase du ticket « retour »**, au moment du **RETURN**, le retour est effectué à l'adresse **0x0804** (instruction 5) qui suit l'appel du module)

C'est là qu'intervient la PILE (STACK) !!!

Avant l'appel du module, l'adresse **0x0804** (qu'on appelle plus généralement **l'adresse de retour** ou **return address**) est sauvegardée automatiquement dans la pile (plus exactement au sommet de cette pile).

Puis le **PC** est modifié pour sauter à la première instruction du module. (**phase ① du ticket « aller »**).

Au moment du **RETURN** (fin du module), cette adresse de retour sera tout aussi automatiquement récupérée (du sommet de la pile) pour être placée dans le **PC**, ce qui assurera le retour à l'instruction qui suit l'appel du module (**phase ③ du ticket « retour »**).

  : mais attention (erreur fréquente) :

NE PAS CONDONDER :

un saut inconditionnel (ici par **GOTO**) ou conditionnel
(voir plus loin)

(pas de possibilité automatique de retour)

≡ ticket d'« aller simple »

et

un appel de module (ici par **CALL**)

≡ ticket d'« aller »

avec **retour systématique** à l'instruction qui suit
l'appel

≡ ticket « retour »

Donc au total pour un **CALL** :

≡ ticket « aller-retour »

③ La pile pour les PIC

Dans **beaucoup** de micro/processeurs/contrôleurs, la pile est une zone située en **RAM** (à l'**extérieur** de la CPU)

Cela nécessite la gestion d'un **pointeur de pile** (**stack pointer**) pour indiquer, à tout moment, l'adresse du sommet de cette pile.

Avec la possibilité, laissée à l'utilisateur, d'utiliser cette pile pour ses propres besoins :

- zone de stockage pour les variables locales,
- passage de paramètres pour les fonctions ...

**Ce qui peut entraîner parfois des problèmes
(en programmation en assembleur)
en cas de mauvaise gestion de cette pile.**

**Dans le cas des PIC,
la pile est dans la CPU.**

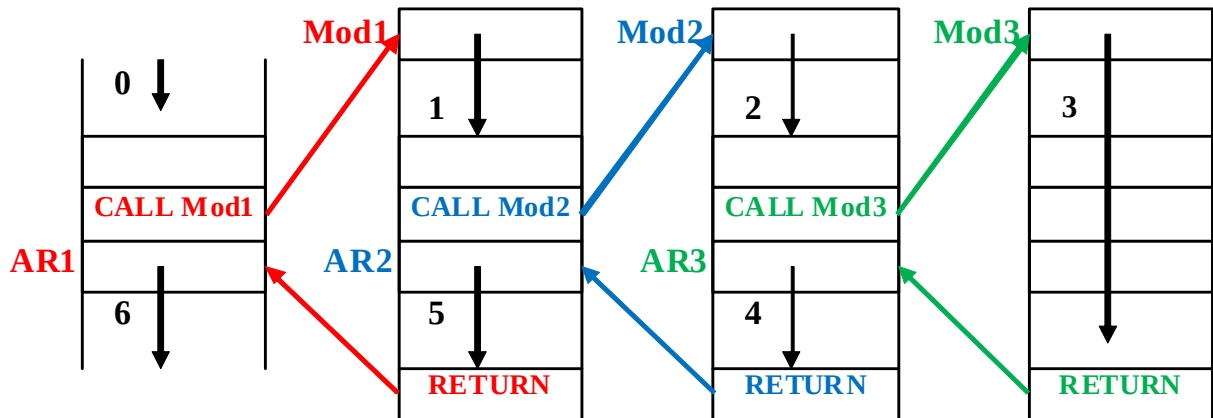
Elle n'est utilisée qu'au moment :

- de l'**appel** des modules :
empilement (sauvegarde) de l'adresse de retour,
- et du **retour** des modules :
dépilement (récupération) de cette adresse de retour.

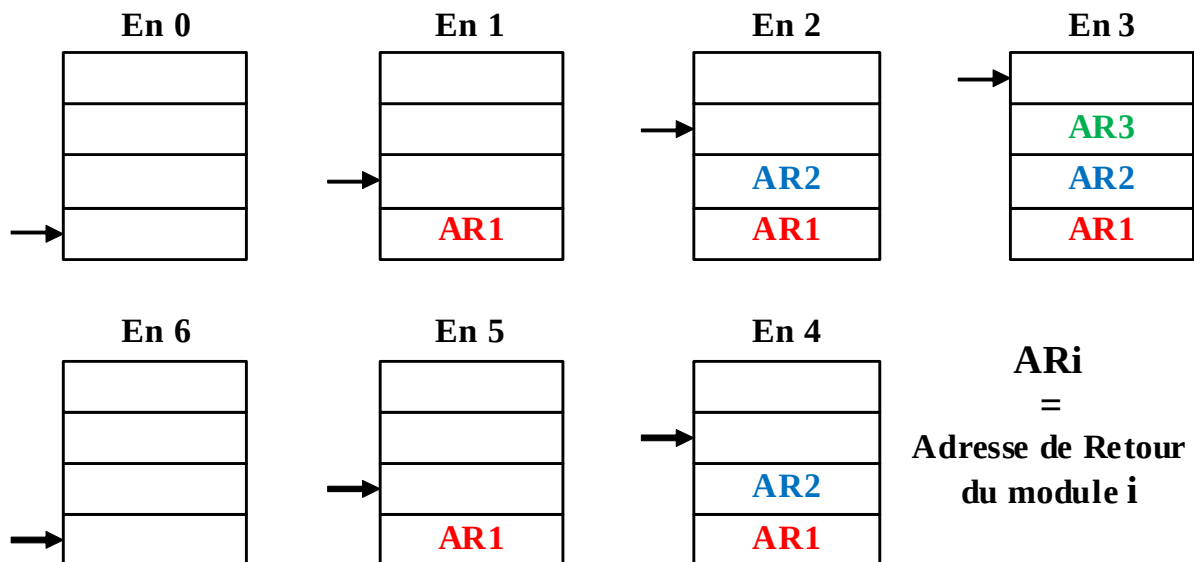
Résumé du cours PIC : chapitre III

La pile est une structure de type **LIFO** (**L**ast **I**n **F**irst **O**ut), comme une « pile d'assiettes » !!!

Ce qui permet les **imbrications** de modules, comme indiqué ci-dessous :



Evolution de la pile au cours des différents appels



Principe d'imbrication des modules

Sur ce schéma, on a pris une pile avec 4 positions, mais pour le PIC 16F877, il y en a 8. On voit ici que si on a plus de 4 imbrications de fonctions, la pile déborde.

Figure III.16. (page 95)

Avantages de la programmation modulaire

- **gain de place en mémoire**

Le module est présent **une seule fois** en mémoire mais peut être exécuté autant de fois que nécessaire.

- programmation **structurée**

Découpage d'un programme **complexe** en **bouts plus simples**.

- meilleure **lisibilité**

Grâce notamment à cette **structuration**.

- **amélioration compréhension** du programme,

Grâce aussi à cette **structuration**.

- **évolutivité** accrue

Plus facile de faire des **modifications** et des **améliorations** sur le programme.

- **tests et mise au point facilités**

Car chaque fonction, qui est toujours plus simple que le main, peut être testée **individuellement** (et parfois **indépendamment** du reste du programme principal).

Résumé du cours PIC : chapitre III

- **conception** générale du programme **plus aisée**

L'assemblage de fonctions, toutes testées individuellement (et à **150%**), transforme la conception du programme principal en « **un jeu de Léo** »,
(rappelez-vous du sable pour faire des briques, des briques pour faire des murs, des murs pour faire des maisons ... Mais pour que vos murs soient solides, il faut que vos briques le soient aussi, pour que votre maison soit solide, il faut que vos murs le soient aussi!!!)

- **réutilisation** possible des fonctions

Certaines fonctions (très **générales** et **génériques**) pourront avantageusement être **réutilisables** dans un autre contexte. Il n'y a qu'à voir la richesse des fonctions disponibles dans les **bibliothèques** du C pour s'en convaincre. De plus, une fonction issue d'une bibliothèque a été en principe validée complètement (c'est une brique solide !!!) : donc si votre programme qui l'utilise ne marche pas, l'erreur vient sûrement de vous ou d'une mauvaise interprétation de la documentation de cette fonction...

Bref beaucoup d'avantages pour l'écriture et la conception de programmes complexes

Une des difficultés dans la conception d'un logiciel applicatif est d'identifier ces fonctions (ou modules) de base qu'il faut écrire puis ensuite tester et enfin rassembler entre elles pour réaliser l'application finale.

Mais, chaque fois que cela sera possible, nous raisonnerons en utilisant des modules :

ce sera un peu notre « **règle d'or** » pendant les TP.

LE JEU D'INSTRUCTIONS

Au total **35 instructions élémentaires**

Le codage de chaque instruction se fait sur **14 bits**
(taille d'une case EEPROM : cf. figure III.1. page 67)

- **6 bits** au maximum réservés pour coder le type d'instruction :
(soit $2^6 = 64$ combinaisons différentes possibles)

champ **OPCODE**

- le reste (**8 bits** au minimum) pour coder l'opérande de l'instruction :
champs **d, FILE, BIT ou LITTERAL**

Classement par **MICROCHIP** en **4 familles**

① instructions **BYTE-ORIENTED**

(orientées **octet : SFR ou GPR**)

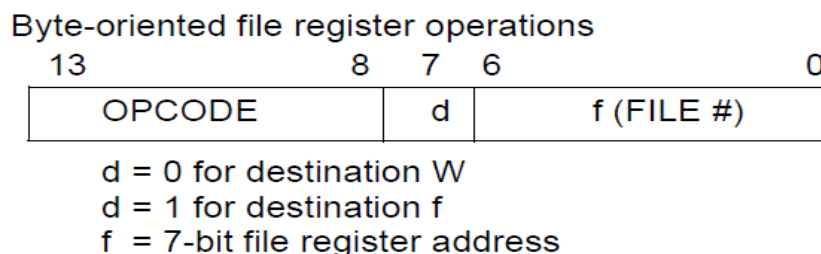


Figure IV.1. (page 101)

② instructions **BIT-ORIENTED**

(orientées **bit**)

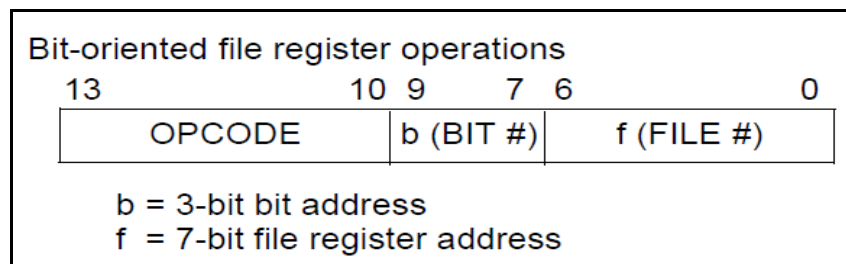


Figure IV.2. (page 102)

LE JEU D'INSTRUCTIONS (suite)

③ instructions **LITTERAL ORIENTED** (orientées **constantes**)

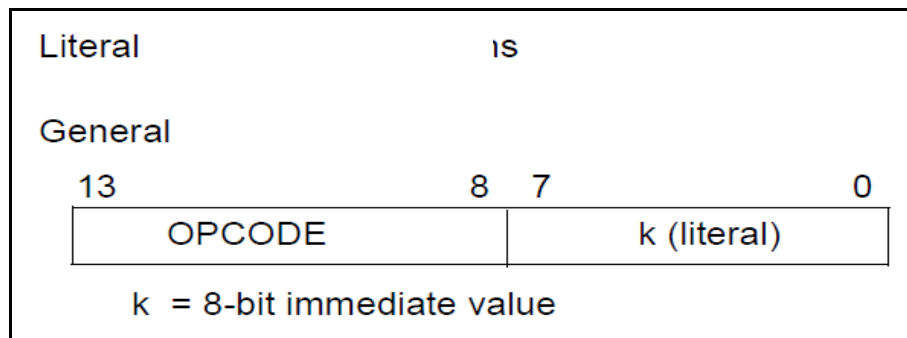


Figure IV.3. (page 102)

④ instructions de **CONTRÔLE**

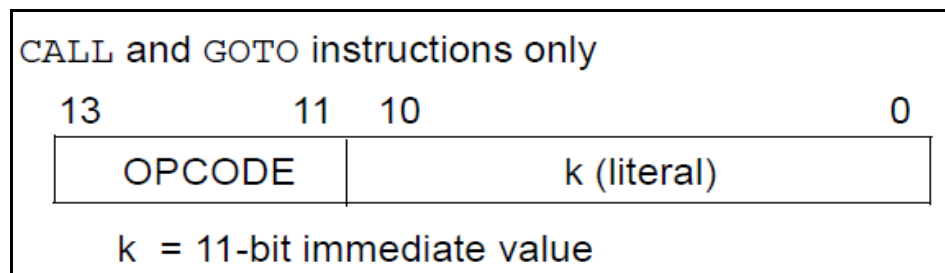


Figure IV.4. (page 102)

Résumé du cours PIC : chapitre IV

Liste complète (fournie par MICROCHIP)

①	②	③	④	⑤	⑥	
Mnemonic, Operands	Description	Cycles	14-Bit Opcode		Status Affected	Notes
			MSb	LSb		
BYTE-ORIENTED FILE REGISTER OPERATIONS						
ADDWF	f, d	Add W and f	1	00 0111 dfff ffff	C,DC,Z	1,2
ANDWF	f, d	AND W with f	1	00 0101 dfff ffff	Z	1,2
CLRF	f	Clear f	1	00 0001 1fff ffff	Z	2
CLRWF	-	Clear W	1	00 0001 0xxx xxxx	Z	
COMF	f, d	Complement f	1	00 1001 dfff ffff	Z	1,2
DECF	f, d	Decrement f	1	00 0011 dfff ffff	Z	1,2
DECFSZ	f, d	Decrement f, Skip if 0	1(2)	00 1011 dfff ffff		1,2,3
INCF	f, d	Increment f	1	00 1010 dfff ffff	Z	1,2
INCFSZ	f, d	Increment f, Skip if 0	1(2)	00 1111 dfff ffff		1,2,3
IORWF	f, d	Inclusive OR W with f	1	00 0100 dfff ffff	Z	1,2
MOVF	f, d	Move f	1	00 1000 dfff ffff	Z	1,2
MOVWF	f	Move W to f	1	00 0000 1fff ffff		
NOP	-	No Operation	1	00 0000 0xx0 0000		
RLF	f, d	Rotate Left f through Carry	1	00 1101 dfff ffff	C	1,2
RRF	f, d	Rotate Right f through Carry	1	00 1100 dfff ffff	C	1,2
SUBWF	f, d	Subtract W from f	1	00 0010 dfff ffff	C,DC,Z	1,2
SWAPF	f, d	Swap nibbles in f	1	00 1110 dfff ffff		1,2
XORWF	f, d	Exclusive OR W with f	1	00 0110 dfff ffff	Z	1,2
BIT-ORIENTED FILE REGISTER OPERATIONS						
BCF	f, b	Bit Clear f	1	01 00bb bfff ffff		1,2
BSF	f, b	Bit Set f	1	01 01bb bfff ffff		1,2
BTFSC	f, b	Bit Test f, Skip if Clear	1 (2)	01 10bb bfff ffff		3
BTFSS	f, b	Bit Test f, Skip if Set	1 (2)	01 11bb bfff ffff		3
LITERAL AND CONTROL OPERATIONS						
ADDLW	k	Add literal and W	1	11 111x kkkk kkkk	C,DC,Z	
ANDLW	k	AND literal with W	1	11 1001 kkkk kkkk	Z	
CALL	k	Call subroutine	2	10 0kkk kkkk kkkk		
CLRWDAT	-	Clear Watchdog Timer	1	00 0000 0110 0100	$\overline{TO}, PD$	
GOTO	k	Go to address	2	10 1kkk kkkk kkkk		
IORLW	k	Inclusive OR literal with W	1	11 1000 kkkk kkkk	Z	
MOVLW	k	Move literal to W	1	11 00xx kkkk kkkk		
RETFIE	-	Return from interrupt	2	00 0000 0000 1001		
RETLW	k	Return with literal in W	2	11 01xx kkkk kkkk		
RETURN	-	Return from Subroutine	2	00 0000 0000 1000		
SLEEP	-	Go into standby mode	1	00 0000 0110 0011	$\overline{TO}, PD$	
SUBLW	k	Subtract W from literal	1	11 110x kkkk kkkk	C,DC,Z	
XORLW	k	Exclusive OR literal with W	1	11 1010 kkkk kkkk	Z	

Figure IV.5. (page 104)

f : adresse (ou nom) d'un SFR ou GPR
d : destination (W si 0, f si 1)

b : bit concerné (0 à 7)
k : valeur du littéral (constante)

Résumé du cours PIC : chapitre IV

Exemple complet de codage machine d'une instruction (fait « à la main » !!!)

INCF PORTA

Cette instruction incrémente le contenu du registre SFR **PORTA** et place le résultat dans ce même registre **PORTA**.

Remarque

Si on avait eu :

INCF PORTA,W

alors le résultat de l'incrémentation du contenu de **PORTA** aurait été mis dans **W** et le registre **PORTA** n'aurait pas été modifié !!!



Figure IV.6. (page 105)

- ici **d = 1** : résultat dans **PORTA** et pas dans **W** !!!
- les 7 bits **fff ffff** du champ **FILE** sont **l'adresse** de l'opérande, ici le registre **PORTA** (en réalité l'offset de cette adresse) :



Ce registre est à l'offset **0x05=0b 0000 0101** de la **bank 0 RAM**

Au final, le code machine sera : **0b 00 1010 1000 0101** soit **0x 0A85**.

**En fait, tout ce travail de codage
(passage mnémonic+opérande vers code machine)
est fait automatiquement par le logiciel d'assemblage !!!**

ET DONC...
CONSEILS A SUIVRE IMPERATIVEMENT

**Il est donc important, dans votre programme source,
(que vous programmiez en assembleur ou en C)
de conserver les mêmes noms pour les SFR que ceux fournis
dans la documentation hardware du PIC,
et de ne pas « inventer » vos propres noms.**

On s’y retrouve mieux et tout est prévu pour que ça marche !!!

**Par contre, vous pouvez choisir les noms
que vous voulez pour vos variables
(stockées en GPR).**

**En essayant quand même de prendre
un nom de variable en rapport avec le rôle
joué par cette variable dans votre programme.**

**Quand un étudiant choisit d’appeler « toto »
une variable qui sert de compteur,
je me dis qu’il n’a pas pris en compte cette remarque.**

**De même, les noms de labels (ou étiquettes)
et les noms des sous-programmes (modules ou fonctions)
doivent aussi être « parlants » pour aider :**
- à la compréhension,
- à la lisibilité,
- à la mise au point ...
de votre programme ...

Autre classement plus efficace des instructions

Le classement précédent de **MICROCHIP** n'est pas opérationnel pour un utilisateur de ces instructions.

Il vaut mieux les classer en fonction du **type de traitement**.

Avec ce nouveau critère, je distingue **5 grandes familles** :

- ① instructions de **transfert de données**
mouvements et déplacements entre registres,
- ② instructions de **traitement arithmétique**
*les additions, les soustractions,
les in(dé)crémentations, les «mises à 0 »,*
- ③ instructions de **traitement logique**
*opérations AND, IOR, XOR, COM,
rotations, permutations, positionnement de bit,*
- ④ instructions de **rupture de séquence**
inconditionnelles et conditionnelles,
- ⑤ **autres instructions**
modes de fonctionnement particuliers...

Dans le tableau suivant, la notation :

{d = f ou W}

spécifie la destination du résultat :

- si **d = f** ou **1** ou **d absent** (par défaut) alors **résultat dans f**,
- si **d = W** ou **d = 0** alors **résultat dans W**.

Mnémonic	Opérande	Description	Flags	Cycle	Page	
① INSTRUCTIONS DE TRANSFERT						
1	MOVLW	k	$k \rightarrow W$		1	110
2	MOVWF	f	$W \rightarrow f$		1	111
3	MOVF	f,d	$f \rightarrow \{d = f \text{ ou } W\}$	Z	1	112
② INSTRUCTIONS DE TRAITEMENT ARITHMETIQUE						
4	ADDLW	k	$k + W \rightarrow W$	C DC Z	1	113
5	ADDWF	f,d	$f + W \rightarrow \{d = f \text{ ou } W\}$	C DC Z	1	114
6	SUBLW	k	$k - W \rightarrow W$	C DC Z	1	115
7	SUBWF	f,d	$f - W \rightarrow \{d = f \text{ ou } W\}$	C DC Z	1	116
8	INCF	f,d	$f + 1 \rightarrow \{d = f \text{ ou } W\}$	Z	1	117
9	DECF	f,d	$f - 1 \rightarrow \{d = f \text{ ou } W\}$	Z	1	118
10	CLRW		$W = 0$	Z = 1	1	119
11	CLRF	f	$f = 0$	Z = 1	1	120
③ INSTRUCTIONS DE TRAITEMENT LOGIQUE						
12	ANDLW	k	$k \text{ AND } W \rightarrow W$	Z	1	121
13	ANDWF	f,d	$f \text{ AND } W \rightarrow \{d = f \text{ ou } W\}$	Z	1	122
14	IORLW	k	$k \text{ IOR } W \rightarrow W$	Z	1	123
15	IORWF	f,d	$f \text{ IOR } W \rightarrow \{d = f \text{ ou } W\}$	Z	1	124
16	XORLW	k	$k \text{ XOR } W \rightarrow W$	Z	1	125
17	XORWF	f,d	$f \text{ XOR } W \rightarrow \{d = f \text{ ou } W\}$	Z	1	126
18	COMF	f,d	$\bar{f} \rightarrow \{d ? f : W\}$	Z	1	128
19	RLF	f,d	rotation gauche de f par C $\rightarrow \{d = f \text{ ou } W\}$	C	1	129
20	RRF	f,d	rotation droite de f par C $\rightarrow \{d = f \text{ ou } W\}$	C	1	130
21	SWAPF	f,d	permuter les 2 quartets de f $\rightarrow \{d = f \text{ ou } W\}$		1	131
22	BCF	f,b	mise à zéro du bit b de f		1	132
23	BSF	f,b	mise à un du bit b de f		1	133
④ INSTRUCTIONS DE RUPTURE DE SEQUENCE						
INCONDITIONNELLES						
24	GOTO	k	saute toujours à l'adresse k		2	135
25	CALL	k	appel du sous-programme k		2	136
26	RETURN		retour de sous-programme		2	137
27	RETLW	k	retour de sous-programme avec $W = k$		2	138
28	RETFIE		retour d'interruption		2	139
CONDITIONNELLES						
29	DECFSZ	f,d	$f - 1 \rightarrow \{d = f \text{ ou } W\}$ et si 0 saute <u>une</u> instruction		1(2)	140
30	INCFSZ	f,d	$f + 1 \rightarrow \{d = f \text{ ou } W\}$ et si 0 saute <u>une</u> instruction		1(2)	143
31	BTFSS	f,b	teste bit b de f et si 1 saute <u>une</u> instruction		1(2)	144
32	BTFSC	f,b	teste bit b de f et si 0 saute <u>une</u> instruction		1(2)	146
⑤ AUTRES INSTRUCTIONS						
33	NOP		No OPération :		1	147
34	CLRWDT		reset du timer du watch dog	$\overline{TO} \quad \overline{PD}$	1	148
35	SLEEP		passage dans le mode « standby »	$\overline{TO} \quad \overline{PD}$	1	150

Figure IV.9. (page 109)

Résumé du cours PIC : chapitre IV

Les **35 instructions** (avec des exemples détaillés en assembleur et des liens avec le langage C) sont détaillées dans le polycopié de cours entre les pages **110** et **150** incluses (cf. dernière colonne du tableau précédent).

En voici quelques unes, parmi les plus significatives, dans chaque famille.

① EXEMPLES D'INSTRUCTIONS DE TRANSFERT

Famille des **MOV**.

Le plus souvent utilisées (dans un programme la moitié des instructions représente des transferts entre les différents registres **SFR** et **GPR**)

⚡ ⚡ **Mais attention de ne pas confondre les 3 types de MOV...**
(voir exercice V.9. page 181 du polycopié de cours pour plus de détails)

1. MOVLW k (comme **MOV** e **L**iteral (**k**) to **W**)

Pour **mettre** le **L**ittéral (**L**) **k** (une **constante**) dans **W**.

Utilisée pour **initialiser** le registre **W** avec une **constante**.

Exemple en assembleur :

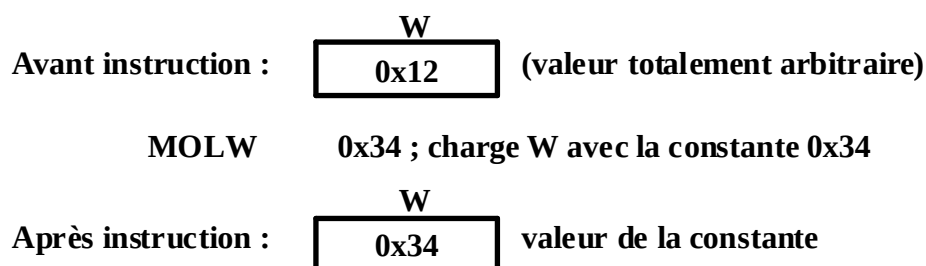


Figure IV.10. (page 110)

Pas d'équivalent direct en C, car pas d'accès direct à **W** avec le langage C.

Résumé du cours PIC : chapitre IV

2. **MOVWF f** (comme **MOV** e **W** to **File** f)

Transfère (copie) le contenu de **W** dans le registre d'adresse **f** de la **RAM** (sens **W** vers **RAM** = cycle **d'écriture** en **RAM**). **W** n'est pas modifié !!!

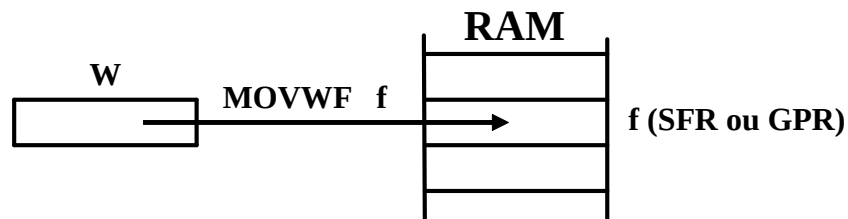


Figure IV.11. (page 111)

Exemple en assembleur :

Dans cet exemple, le **PORTB** est supposé configuré en sortie :

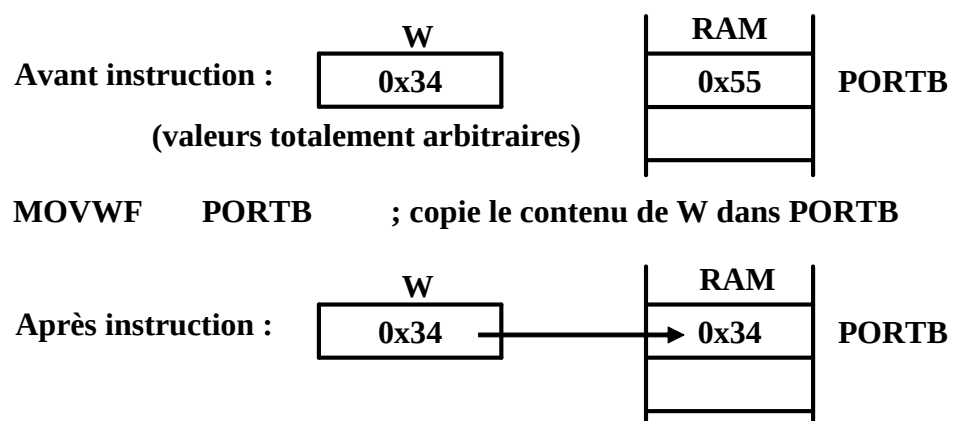


Figure IV.12. (page 111)

On ne peut directement initialiser un **SFR** ou un **GPR** par une constante. Il faut passer par **W** en chargeant d'abord cette constante dans **W**.

Correspondance avec le langage C :

Les 2 instructions suivantes pour écrire dans le **PORTB** (configuré en sortie)

```

MOV LW 0x34      ; constante 0x34 dans W
MOVWF PORTB     ; copie W dans PORTB
```

seront **équivalentes** à la ligne en C suivante :

→ **PORTB = 0x34 ; // sort 0x34 sur le PORTB**

Résumé du cours PIC : chapitre IV

3. **MOVE** **f,d** (comme **MOV**e **F**ile **f** to **d**)

Transfère (copie) le contenu du registre d'adresse **f** de la RAM dans la destination **d**.

Si $d = 0$ (ou **W**), alors la destination est le registre **W**.
(sens **RAM** vers **W** = cycle **de lecture** de la **RAM**)

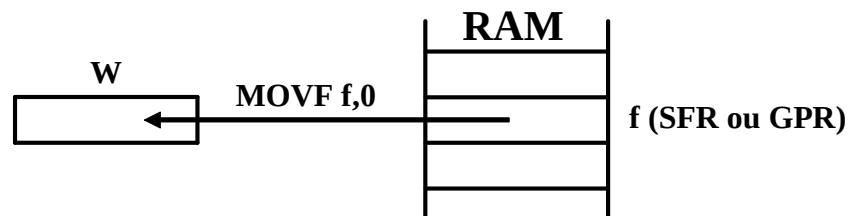


Figure IV.13. (page 112)

Exemple en assembleur :

Dans cet exemple, le **PORTB** est supposé configuré en entrée :

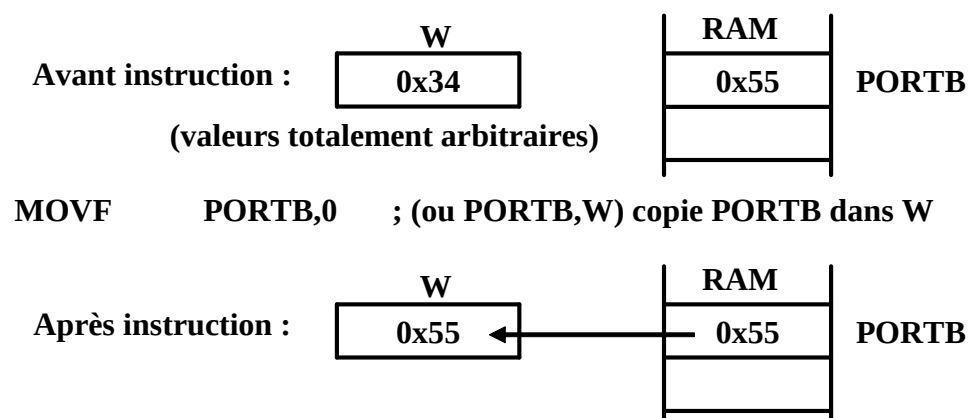


Figure IV.14. (page 112)

Résumé du cours PIC : chapitre IV

Correspondance avec le langage C :

Les 2 instructions suivantes (**input** est une variable stockée dans un **GPR**) permettent de lire le **PORTB** (configuré en entrée) dans la variable **input** :

```
input      :      1      ; réservation de 1 octet en RAM pour  
                        ; la variable (en GPR) input
```

...

```
    [ MOVF  PORTB,W      ; copie PORTB dans W  
      MOVWF input        ; copie W dans la variable input
```

seront **équivalentes**, en langage C, à :

```
    unsigned char input ;    // variable input de type char (1 octet)
```

...

```
    input = PORTB ;        // lecture du PORTB dans input
```

Remarque interessante

Une instruction comme :

MOVF PORTB ; (ou **PORTB,1** ou **PORTB,f**)

semble inutile puisqu'elle copie **PORTB** (ou tout autre registre) sur lui-même, ce qui à priori ne sert à rien.

En réalité, cette instruction va positionner le flag **Z**, ce qui permettra par exemple de le tester **juste après** pour savoir si le contenu de n'importe quel registre (ici le registre **PORTB**) est nul ou pas !!!

② EXEMPLES D'INSTRUCTIONS DE TRAITEMENT ARITHMETIQUE

Famille des **ADD**, **SUB**, **INC**, **DEC** et **CLR**.

4. **ADDLW k** (comme **ADD Literal (k)** and **W**)

Additionne le littéral (**L**) **k** (une **constante**) au contenu du registre **W**, le résultat de cette addition étant mis dans **W**.

Exemple en assembleur :

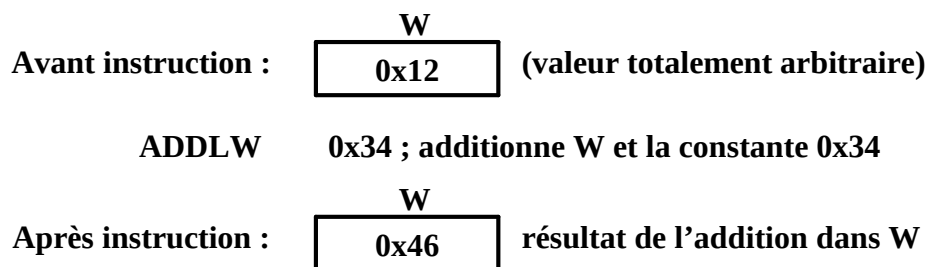


Figure IV.15. (page 113)

Les flags **C**, **DC** et **Z** seront positionnés suivant le résultat obtenu dans **W**.

Rappel

- si résultat **nul** alors **Z** = 1,
- si résultat ≥ 256 (ne tient pas sur un octet) alors **C** = 1,
- si résultat < 256 (tient sur un octet) alors **C** = 0,
- si retenue entre **bit3** et **bit4** alors **DC** = 1 sinon **DC** = 0.

Pas d'équivalent direct en C, car pas d'accès direct à **W** avec le langage C.

Résumé du cours PIC : chapitre IV

5. **ADDWF f,d** (comme **ADD W** and **File f** to **d**)

Additionne les contenus de **W** et de **f** et **place** le résultat :

- soit dans **f** si **d = 1** (ou **d = f** ou si **d** est **omis**),
- soit dans **W** si **d = 0** (ou si **d = W**).

Exemple en assembleur :

Dans cet exemple, i est une variable (en **GPR**) de taille 1 octet :

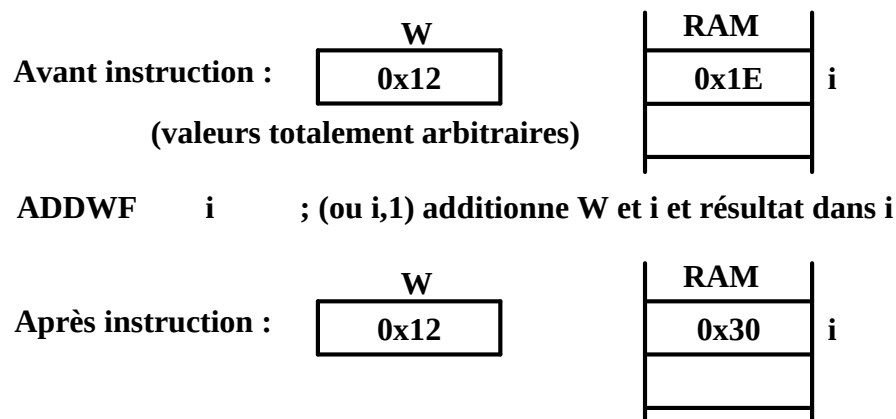


Figure IV.16. (page 114)

Les flags **C**, **DC** et **Z** seront positionnés suivant le résultat de cette addition et comme indiqué dans le rappel de la page précédente.

Correspondance avec le langage C :

L'exemple précédent s'écrirait tout simplement en C :

```
i = i + 18;    // additionne 18 (0x12) à i
```

ou

```
i+ = 18       // idem
```

③ EXEMPLES D'INSTRUCTIONS DE TRAITEMENT LOGIQUE

Famille des **AND**, **IOR**, **XOR**, **COM**, **RxF**, **SWAP** et **BxF**.

12. **ANDLW k** (comme **AND** Literal (**k**) with **W**)

Pour faire un **ET logique** « bit à bit » entre le littéral (**L**) **k** (une **constante**), et le contenu du registre **W**, le résultat de cette opération étant mis dans **W**. Le littéral **k** est souvent dénommé un **masque**.

Exemple en assembleur :

Contenu de **W** et valeur du masque (**0xF0**) représentés en binaire.

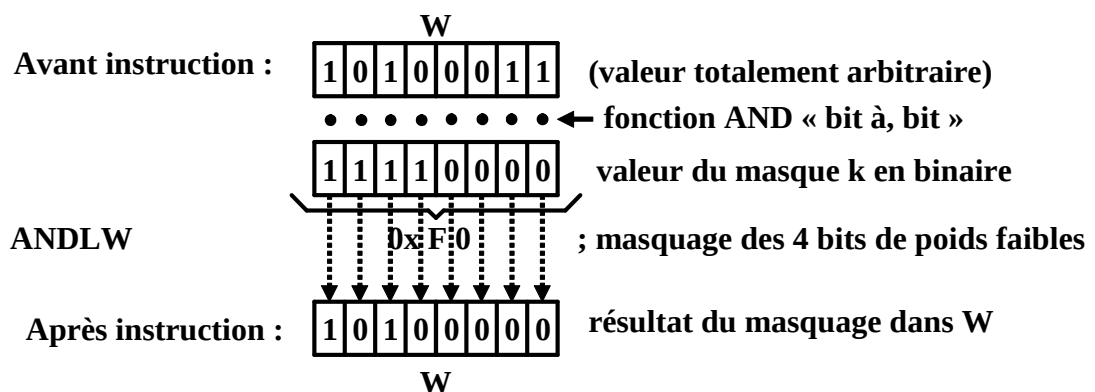


Figure IV.23. (page 121)

- bits de **W** de même poids que bits à **1** du masque **inchangés**,
- bits de **W** de même poids que bits à **0** du masque mis à **0**,

Un ET « bit à bit » permet donc de forcer à 0 certains bits sans toucher les autres, en fonction de la valeur choisie pour le masque.

Seul le flag **Z** sera positionné suivant le résultat obtenu dans **W**.

Pas d'équivalent direct en C, car pas d'accès direct à **W** avec le langage C.

Résumé du cours PIC : chapitre IV

13. **ANDWF f,d** (comme **AND W** with **File f** to **d**)

Fait un **ET logique** « bit à bit » entre contenus de **W** et de **f** et résultat :

- soit dans **f** si **d = 1** (ou **d = f** ou si **d** est **omis**),
- soit dans **W** si **d = 0** (ou si **d = W**).

Exemple en assembleur :

Le masque est ici dans **W** ce qui permet (contrairement au cas précédent) de faire des masques variables et le résultat du masquage est mis dans **f**.

(détection de la parité en ne gardant que le **LSB**)

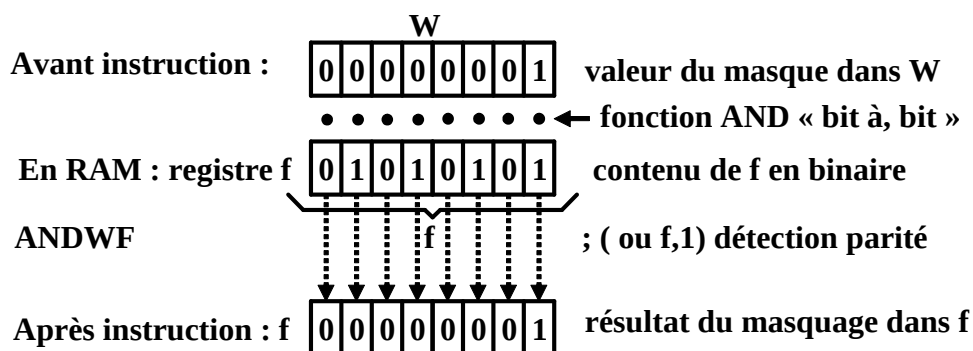


Figure IV.24. (page 122)

Correspondance avec le langage C :

Les 2 instructions (pour passer du code **ASCII** au codage **DCB** sur 1 octet)

car **DB** 0x35 ; car initialisée avec code ASCII de 5

...

MOVLW 0x0F ; masque quartet fort dans W
ANDWF car ; ASCII vers **D**écimal **C**odé **B**inaire

seront **équivalentes**, en langage C, à :

unsigned char car = '5'; // variable car contenant code ASCII de 5 = 0x35

...

→ car = car & 0x0F ; // masquage de car par opérateur « bit » &

Résumé du cours PIC : chapitre IV

Famille des **positionnements de bit** : BCF et BSF.

22. **BCF** **f,b** (comme **Bit Clear in File** (**f**) : bit **b**)

Instruction orientée bit pour **mettre à 0** le bit **b** (de **0** à **7**) du file **f**.

**Très fréquemment utilisée car souvent,
on ne doit modifier qu'un seul bit dans un SFR du PIC.**

Exemple et correspondance avec le langage C :

L'instruction suivante permettant de forcer à **0** la **broche 7** du **PORTB**, c'est-à-dire la broche **RB7** (configurée préalablement en sortie) :

BCF PORTB,7 ; met à 0 V la **ligne 7** du **PORTB**

sera équivalente à la ligne en C suivante :

→ **RB7 = 0 ;** // met à 0 V la **ligne 7** du **PORTB**

Remarque

Le couple compilateur/linker du C assure, grâce à la ligne :

#include <pic.h>

la correspondance entre :

les noms des broches (ici **RB7**) et la **ligne 7** du **PORTB**.

Et ceci est vrai pour tous les registres **SFR** et pour tous leurs bits individuels, en gardant bien sûr les mêmes noms que ceux fournis dans le datasheet du PIC (**ne pas inventer ses propres noms !!!**).

**Le fichier d'inclusion pic.h permet donc de faire le lien
entre le hardware (mapping) et le software.**

④ EXEMPLES D'INSTRUCTIONS DE RUPTURE DE SEQUENCE

Pour « **briser** » le déroulement séquentiel d'un programme.

INSTRUCTIONS DE RUPTURE INCONDITIONNELLE

Elles dureront donc toujours **deux cycles** :

- le premier pour **vider le pipe**,
- le deuxième pour **effectuer la rupture** de séquence demandée.

24. GOTO k (comme **GOTO** label **k**)

(Déjà vue et citée pour mémoire)

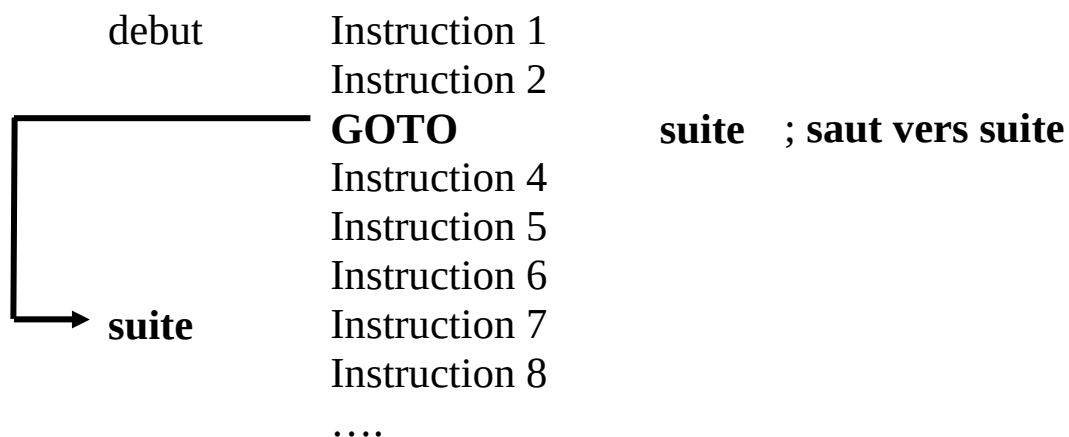


Figure IV.36. (page 135)

- **GOTO** permet **de sauter à une autre adresse (avant ou après)**,
- pas de retour possible vers l'endroit de l'appel : « **ticket aller simple** »,
- grâce au système des labels, on peut donner un nom **symbolique** à k,

Remarque en langage C :

Existe aussi en langage C, mais il est déconseillé de l'utiliser au sein d'un programme écrit en C.

Utiliser plutôt les instructions **while, for...** (plus « structurantes »)

INSTRUCTIONS DE RUPTURE INCONDITIONNELLE

25. **CALL** **k** (comme **CALL** subroutine **k**)

(Déjà vue et citée pour mémoire pour ne pas la confondre avec **GOTO**)

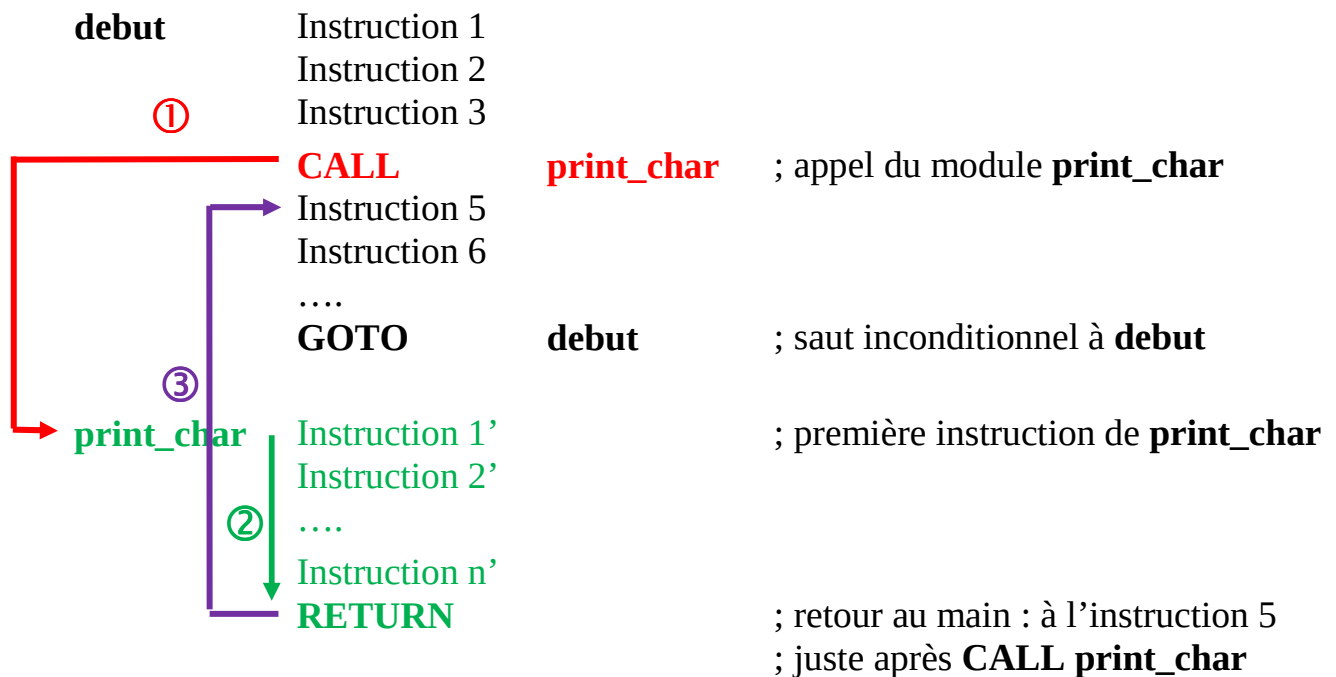


Figure IV.37. (page 136)

- **CALL** permet de sauter à une autre adresse (avant ou après) (phase ① = « ticket aller »),
- le module appelé s'exécute alors (phase ②),
- puis au moment du **RETURN** (fin du module) retour vers l'instruction qui suit l'appel du module (phase ③ = « ticket retour »),
- grâce au système des labels, on peut donner un nom **symbolique** à k.

Correspondance avec le langage C :

L'exemple précédent pourrait s'écrire en C :

```
while(1)                      // boucle sans fin à cause du GOTO debut
{
    ...
    print_char();      // appel de la fonction print_char (ici sans paramètre)
    ...
}
```

INSTRUCTIONS DE RUPTURE CONDITIONNELLE

La rupture ne se fera que **si une condition est réalisée** (durée **2 cycles**).

Sinon, le programme continue en passant à l'instruction qui se trouve dans la case mémoire suivante (déroulement séquentiel et durée **1 cycle**).

**Particulièrement adapté pour implanter
des boucles ou des tests conditionnels.**

Famille : **DECFSZ** et **INCFSZ**.

29. DECFSZ f,d (comme **DEC**rémente **F**ile (**f** to **d**) and **S**kip if **Z**ero)

Cette instruction se déroule en **deux temps** :

- **dérémentation** du contenu du registre **f** et résultat dans **d**,
- **test du résultat** de la décrémentation :
- **si nul** : **saute l'instruction suivante** en mémoire (une seule sautée)
 (rupture conditionnelle de séquence et durée 2 cycles),
- **si non nul** : passage à **l'instruction suivante** en mémoire
 (déroulement séquentiel et durée 1 cycle).

Exemple en assembleur :

	MOVLW MOVWF	10 compteur
debut_dans_boucle	Instruction 1 dans boucle Instruction 2 dans boucle DECFSZ GOTO	compteur debut_dans_boucle
debut_hors_boucle	Instruction 1 hors boucle Instruction 2 hors boucle ...	

Figure IV.38. (page 140)

Résumé du cours PIC : chapitre IV

INSTRUCTIONS DE RUPTURE CONDITIONNELLE (suite)

Organigramme du programme précédent :

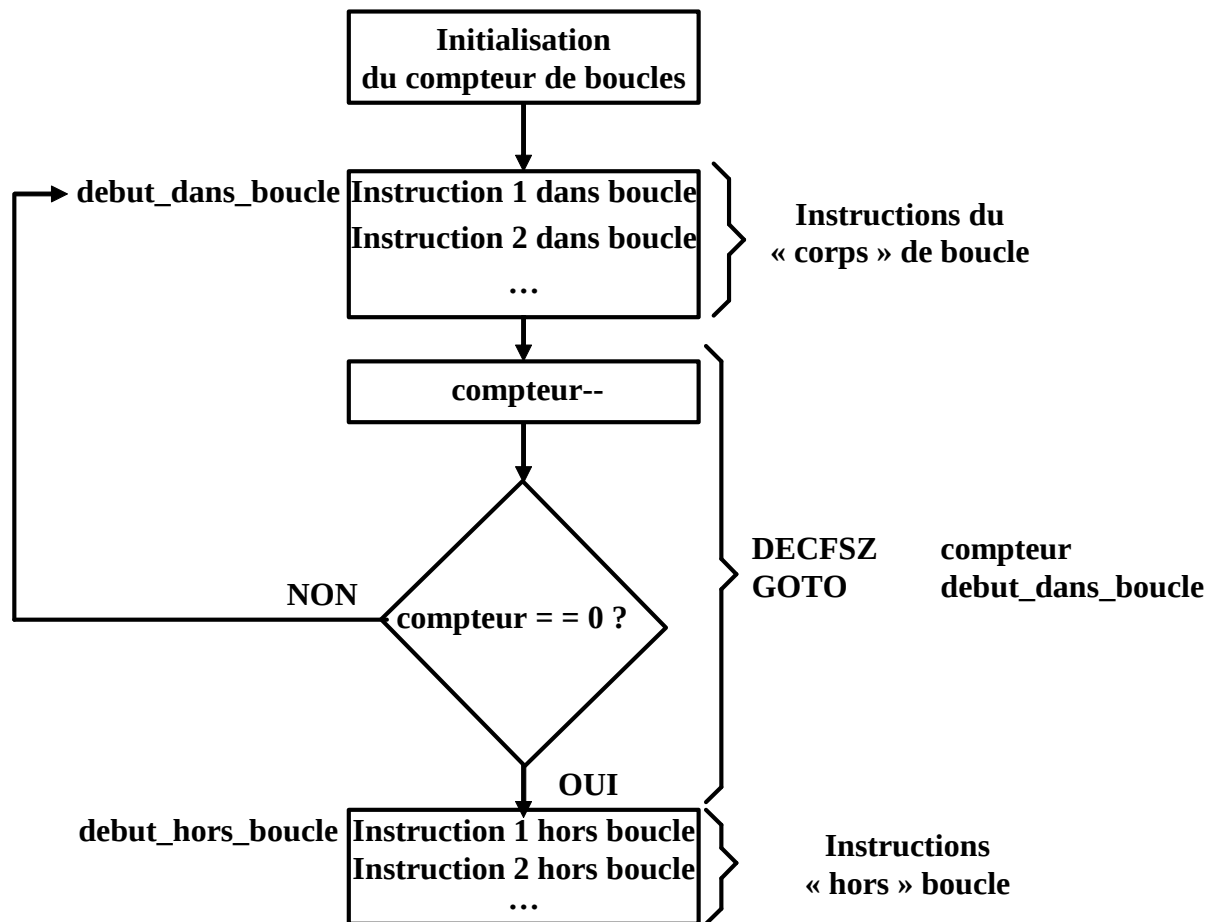


Figure IV.39. (page 141)

Remarque :

Si en assembleur, l'approche graphique offerte par un organigramme a du sens, en langage évolué, comme en C, à cause de la structuration naturelle offerte par les instructions de contrôle de haut niveau, on peut l'éviter.

Par contre, et ceci est une remarque générale, lorsqu'un organigramme est trop important (par exemple si sa taille dépasse une page) il est déconseillé de l'utiliser, car il existe d'autres méthodes (comme la programmation modulaire) qui sont mieux adaptées dans ce cas.

INSTRUCTIONS DE RUPTURE CONDITIONNELLE (suite)

Quelques remarques et pièges à éviter

Cette boucle sera ici répétée **10 fois**.

Mais attention, si on initialise **compteur** avec **0**, elle sera répétée **256** fois, car on **décrémente** la variable **compteur** avant de la tester...

Autre type d'erreur à éviter :

Si on écrit :

DECFSZ compteur, **W** ; résultat décrémentation dans **W**

au lieu de :

DECFSZ compteur ; résultat décrémentation
 ; dans **compteur**

la variable **compteur** ne sera pas décrémentée
et on ne sortira jamais de cette boucle !!!

Correspondance avec le langage C :

Le programme précédent s'écrirait en C (boucle **do...while**) :

```
compteur = 10;           // initialisation du compteur de boucles
do
{
....                     // instructions de la boucle
compteur-- ;
}
while (compteur != 0) ;
```

Résumé du cours PIC : chapitre IV

INSTRUCTIONS DE RUPTURE CONDITIONNELLE (suite)

Famille : **BTFSS** et **BTFSC**.

31. BTFSS f,b (comme **Bit Test (b) in File (f) and Skip if Set**)

Orientée bit pour **tester** le bit **b** (de **0** à **7**) de **f** et :

- **si b = 1** : saute l'instruction suivante en mémoire (une seule sautée) (rupture conditionnelle de séquence et durée 2 cycles),
- **si b = 0** : passage à l'instruction suivante en mémoire (déroulement séquentiel et durée 1 cycle).

**Très fréquemment utilisée car souvent,
on ne doit tester qu'un seul bit dans un des SFR du PIC.**

Par exemple, pour attendre qu'un bit **b**, normalement à **0**, d'un **SFR f**, passe à **1**.

Ce passage de **0** à **1** indique souvent qu'un évènement (lié au hardware intérieur ou extérieur) a eu lieu, et qu'on peut effectuer le traitement correspondant à cet évènement.

Les exemples sont nombreux (liste pas du tout exhaustive...) :

- un **caractère** vient d'arriver sur une liaison **série** : il faut le lire,
- un **timer** (compteur de temps) a atteint une valeur définie : il faut faire une action donnée comme incrémenter un compteur de secondes par exemple,
- une **broche d'un port en entrée** est passée de **0** à **1** par appui sur un interrupteur : il faut effectuer l'action prévue consécutive à cet appui,
- un **convertisseur analogique/numérique** a fini sa conversion : il faut lire la valeur numérique du résultat ...

*Ces différents cas peuvent être traités efficacement (mais un peu plus difficilement) par la technique des **interruptions**, comme on le verra plus loin en Travaux Pratiques).*

Résumé du cours PIC : chapitre IV

INSTRUCTIONS DE RUPTURE CONDITIONNELLE (suite)

Mais il existe une autre façon de les gérer, plus facile à mettre au point, si les **contraintes de temps réel ne sont pas critiques** ou **primordiales**.

Cette technique s'appelle la « **scrutation** » (ou « **pooling** »).

Elle consiste à surveiller (**scruter**) l'état du bit **b** concerné et à attendre qu'il passe à **1**.

On cherche en fait à **synchroniser** le programme avec cet évènement, matérialisé ici par le **front montant** du bit **b** lors de son passage de **0** à **1**.

Les deux lignes suivantes permettent de réaliser cette synchronisation :

synchro	BTFSS	f,b	; bit b de f = 0 ?
	GOTO	synchro	; oui, alors reboucler sur le
			; label synchro et continuer
			; à attendre
bit_b_a_1			; suite du programme :
			; à partir de là, le bit b de f est
			; passé à 1.

Figure IV.40. (page 145)

Avant t_1 , le programme reboucle en permanence sur synchro.

A t_1 , quand **b** passe à **1**, l'instruction :

GOTO synchro

est sautée **et le programme peut continuer au label bit_b_a_1.**

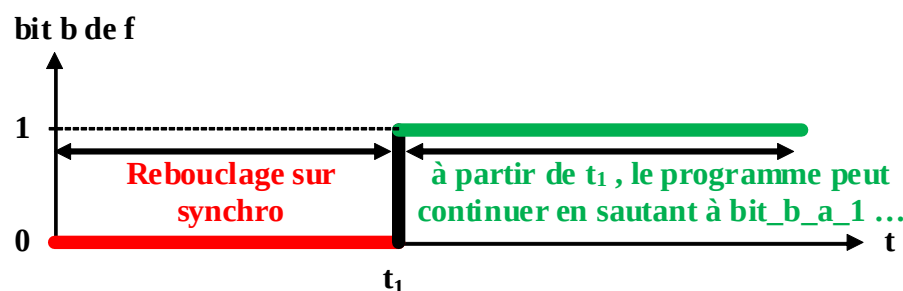


Figure IV.41. (page 145)

Résumé du cours PIC : chapitre IV

INSTRUCTIONS DE RUPTURE CONDITIONNELLE (suite)

Exemple : comment détecter l'appui sur un bouton poussoir **BP** ?

La **broche 7** du **PORTB** est supposée fonctionner en entrée.

Elle est câblée sur un bouton poussoir **BP** comme indiqué ci-dessous :

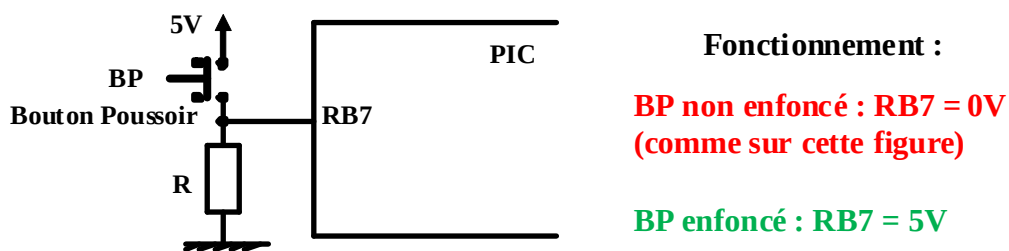


Figure IV.42. (page 145)

Dès qu'on appui sur le **BP**, la broche d'entrée **RB7** passe de **0** à **1**.

Codage de détection de l'appui sur BP en assembleur :

```
attente_appui  BTFSS  PORTB,7      ; bit 7 de PORTB == 0 ?  
                GOTO   attente_appui ; oui, alors reboucler  
                                           ; sur attente_appui  
  
appui_BP_OK    .....                ; à partir de là, BP enfoncé
```

Codage de détection de l'appui sur BP en langage C :

Cette même application donnerait en C :

```
while (RB7 == 0) ;    // tant que RB7==0 (pas d'appui)  
                      // attente d'un appui sur BP  
...                   // à partir de là, BP est enfoncé
```

ou :

```
while (!RB7) ;       // idem : tant que RB7==0, on attend  
...                   // à partir de là, BP est enfoncé
```

DIRECTIVES D'ASSEMBLAGE

Si on ne prend pas un **minimum de précautions**, la programmation en assembleur peut vite devenir « **incompréhensible** » et « **illisible** » surtout lorsque la taille des programmes augmente.

A cause surtout de la présence de trop de valeurs en **hexadécimal** :

- constantes,
- adresses de registres **SFR**,
- adresses de variables **GPR**,
- valeur des registres ou des variables...

Pour cela, il existe un certain nombre d'aides à l'écriture de programmes :

les directives d'assemblage.

Ce sont des **mots réservés** du langage, constitués d'une suite de **quelques lettres** permettant de rendre les sources « **symboliques** ».

Attention :

Même si les directives d'assemblage et les mnémonics sont dans la même colonne d'un fichier source, il ne faut pas les confondre.

- les directives d'assemblage « ne sont pas vues » par le PIC mais sont gérées par le logiciel d'assemblage (en amont de l'exécution).

Par contre :

- les mnémonics sont « vues » par le PIC (sous leur forme code machine) car elles correspondent aux instructions élémentaires qui seront exécutées par celui-ci.

DIRECTIVES D'ASSEMBLAGE (suite)

Ces directives dépendent du **logiciel d'assemblage**.

En TP, on utilisera l'assembleur **MPASM** (**MPLINK** et **MPLIB**) fourni avec l'Environnement de Développement Intégré **MPLAB IDE**, le tout disponible **gratuitement** sur le site de **MICROCHIP**.

Télécharger MPLAB pour vous « muscler » chez vous !!!

DIRECTIVE #INCLUDE

Pour insérer un fichier **d'inclusion complémentaire** dans votre source. (qui évite d'avoir à connaître ou retrouver les adresses des **SFR**)

Syntaxe et exemple :

#include <p16f877a.inc> ; fichier contenant la correspondance entre
; le nom symbolique des **SFR** et leur **adresse**
; ainsi que **certains de leurs bits**.

Quelques extraits de ce fichier pour bien comprendre :

```
; repérage des SFR par leur adresse dans le mapping
...
STATUS EQU    H'0003' ; adresse de STATUS : chaque fois
                    ; que le mot STATUS sera utilisé
                    ; MPASM le remplace par 0x03
FSR    EQU    H'0004' ; adresse de FSR
PORTA  EQU    H'0005' ; adresse de PORTA ...
...
; repérage des bits de STATUS
; utile pour les instructions « orientées bit »

IRP    EQU    H'0007' ; position bit IRP (bit7)
RP1    EQU    H'0006' ; position bit RP1 (bit6)
RP0    EQU    H'0005' ; position bit RP0 (bit5)
```

Figure IV.45. (page 152)

Résumé du cours PIC : chapitre IV

DIRECTIVE EQU

Pour déclarer et définir des **constantes** (sans réservation mémoire en **GPR**).
Directive à associer à l'adressage Littéral.

Syntaxe assez simple :

NOM_CSTE EQU VALEUR

① les **constantes** « **hardware** » : ce sont le plus souvent des adresses.

Exemples :

Les adresses des **SFR** déclarées par des **EQU** dans le fichier **p16f877a.inc**.

On pourrait rajouter les lignes suivantes pour définir les broches du **PORTB**:

RB0	EQU	0	; bit 0 du PORTB
RB1	EQU	1	; bit 1 du PORTB
RB2	EQU	2	; bit 2 du PORTB
...			

② les **constantes** « **software** » (liées à votre programme):
masques, valeurs initiales de compteurs ou de variables...

Exemple :

NOTE_MAX EQU 20

Chaque fois que **NOTE_MAX** (nom de la constante) sera rencontré dans le fichier source, **MPASM** le remplacera par la valeur décimale **20** (=0x14).

Ainsi, une instruction comme :

MOVLW NOTE_MAX

sera alors reconnue et vue par **MPASM** comme (et aura le même effet que) :

MOVLW 20

**Mais l'utilisation de noms symboliques
permettra d'améliorer la lisibilité de votre source.**

DIRECTIVE END

Pour indiquer la **fin du code à assembler**.

Si on rajoute des instructions après le **END**, elles ne seront pas compilées.

Si on oublie cette directive à la fin de son programme, on aura une erreur de compilation.

Les directives suivantes concernent les **variables** d'un programme.

A la base, il existe **trois types de variables globales** :

- les variables **globales non initialisées** situées en **RAM**,
- les variables **globales initialisées** situées aussi en **RAM**,
- les variables **globales (semi)permanentes** en **EEPROM de données**.

Au final, un fichier **source** écrit en assembleur sera une suite :

- de réservation/déclaration mémoire pour ces **variables globales** : en **RAM** ou en **EEPROM de données**,
- de lignes d'instructions assembleur pour le **code du programme** en **EEPROM de code**.

  **Attention : erreur fréquente et récurrente**

Ne pas mélanger ces 2 zones :

pas de déclaration de variables globales dans les lignes de code

pas de lignes de code dans la zone des variables globales

DIRECTIVE CBLOCK/ENDC

Pour les **variables globales non initialisées** situées dans n'importe quelle case **GPR** des **banks 0 à 3** de la RAM.

Je **conseille** de les placer en **zone commune** à toutes les banks :

située dans chaque bank entre les offsets **0x71** et **0x7F**

(la case d'adresse 0x70 étant utilisée par le boîtier ICD3 pour le débogage)

Mais bien sûr, ce n'est pas une obligation !!!

Mais cela évite d'avoir à gérer leur adressabilité, via les bits **RP1** et **RP0** du **SFR STATUS**, puisque **quelque soit** la valeur de ces bits, elles seront toujours accessibles.

Les variables globales non initialisées doivent être déclarées :

- dans une **zone de stockage encadrée** par le **couple de directives CBLOCK/ENDC**,
- en indiquant **leur nom** et **leur taille** en nombre d'octets.

Au démarrage de votre programme, il y aura **n'importe quoi** dans ces variables non initialisées situées en **RAM** (contenu **aléatoire**).

DIRECTIVE CBLOCK/ENDC (suite)

→ **CBLOCK** ADRESSE ; ouverture (début) zone de déclaration
...
→ **ENDC** ; fermeture (fin) zone de déclaration

Syntaxe et exemple :

→ **CBLOCK 0x71** ; **ouverture** zone données en zone commune
; de la **RAM** à partir de l'adresse **0x071**
; déclaration des variables par :
; **nom_variable** : **taille_variable (en nombre d'octets)**
compteur : **1** ; variable **compteur** à l'adresse **0x71**
; de taille **1** octet
tableau : **8** ; **tableau** de taille **8** octets
; premier élément du tableau à l'adresse **0x72**
; deuxième élément du tableau à l'adresse **0x73**
; ...
car : **1** ; variable **car** de taille **1** octet à l'adresse **0x7A**
i : **2** ; **i** est un entier court sur 16 bits (2 octets)
; aux adresses **0x7B** et **0x7C**
...
→ **ENDC** ; **fin** de la zone de données non initialisées

DIRECTIVES DB et DW

Pour donner une valeur **initiale** à des variables (ces variables initialisées auront donc déjà ces valeurs initiales **avant** que votre programme n'ait commencé à s'exécuter) mais pour cela, il faut rajouter un petit programme d'initialisation (**copy_init_data**).

les variables initialisées peuvent aussi être initialisées par les premières instructions de votre programme.

Exemple :

Au lieu de faire la déclaration suivante :

```
tempo    DB        0x05    ; variable tempo de 1 octet initialisée  
                                ; à 0x05 avant exécution du programme
```

on déclarera la variable tempo en zone des variables non initialisées



```
CBLOCK    0x71  
tempo      :    1    ; variable tempo à l'adresse 0x71 taille 1 octet  
                                ; par défaut elle vaudra 0 au reset  
...  
ENDC
```

On écrira, **au tout début** du programme, les deux instructions :

```
MOVLW    0x05    ; charge W avec le littéral  
                                ; (constante) 0x05  
  
MOVWF    tempo   ; copie le contenu de W  
                                ; dans le GPR tempo  
                                ; fin initialisation de tempo
```

AUTRES DIRECTIVES INTERESSANTES

Pour assurer l'adressabilité des SFR et des GPR de la RAM,

DIRECTIVE BANKSEL : adressage direct

Rappel

Avant d'accéder à un SFR ou un GPR, on sait qu'il faut toujours que les bits RP1 et RP0 de STATUS permettent de sélectionner la bank où se trouve ce SFR ou ce GPR qu'on souhaite lire ou modifier.

BANKSEL place automatiquement ces bits RP1 et RP0 en fonction de la bank où se trouve ce SFR ou ce GPR (connue par MPASM à l'assemblage).

Syntaxe et exemple :

Supposons que la variable k se trouve en bank 3 (RP1=1 et RP0=1).

Alors, si on écrit :

BANKSEL k ; sélection de la bank où est k
MOVWF k ; copie W dans k

MPASM remplace la première ligne par les deux lignes suivantes :

BSF STATUS,RP1 ; mise à 1 de RP1
BSF STATUS,RP0 ; mise à 1 de RP0

Ce qui permettra ensuite de bien copier W dans k !!!

Et pas dans une case d'une autre bank ayant le même offset que k !!!
(ce qui arriverait si on oubliait de faire **BANKSEL** k avant de faire référence à la variable k)

Synthèse et canevas proposé :

le fichier vierge_asm.asm

présent sur les PC des labos et à utiliser comme base de départ dans tout nouveau projet